

Lesson 1 of 11

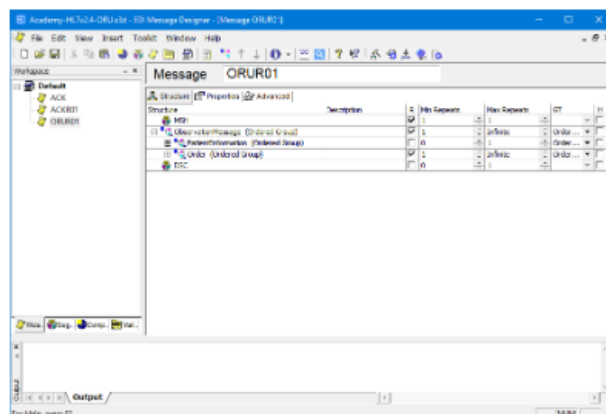
Message Handling in Rhapsody

The Rhapsody Messaging Toolkit (EDI Message Designer, EDI Explorer and Map Designer) allows Rhapsody to interpret messages other than XML and JSON. These tools make it possible for message components to be identified and addressed, either for reading and extracting values, or for writing and updating values.

To interpret a message, the message must have a known structure (for example, fixed-width, CSV, HL7 v2) with the ability to create a definition for that structure to be used by the engine. The EDI Message Designer allows you to define the structure to be as rigid (such as for fixed-width message structures) or as flexible (for example, with separators defined within the message) as you need it to be.

This example shows an HL7 v2.4 message definition in the EDI Message Designer. It includes:

- A generic acknowledgement (ACK) message.
- An observation results (ORUR01) message.
- An observation acknowledgement (ACKR01) message.



\Message Parsing

For a simple route, where no message definition has been associated to it, no parsing occurs and only the default message properties are associated with each message as it begins

processing (**InputTime**, **rhapsody:InputCommunicationPoint** and **rhapsody:InputCommunicationPointId**).

When a definition is attached to a route, parsing will occur when the message reaches the first filter. If two or more definitions are attached to a route, Rhapsody will attempt to parse the message against each of the definitions at the same time. This increases the processing load. Where the definitions are similar (for example, HL7 definitions differing by version number), it makes it impossible to know which definition will be used to parse a particular message.

✎ We recommend that no more than one message definition is attached to a route.

When an HL7 message definition is attached to a route and parsing is triggered by a filter, additional properties are associated with the messages:

Message Properties ▾

Property Name	Property Value
DefinitionName 	Academy-HL7v2.4-ORU.s3d
InputTime	1596058145442
MessageName 	ORUR01
rhapsody:InputCommunicationPoint	Incoming Messages
rhapsody:InputCommunicationPointId	58
rhapsody:InputMessageType 	ORUR01
rhapsody:OutputMessageType 	ORUR01

Showing 7 of 7

Rhapsody holds a reference to the parsed content of a message.

- If the parsed content of a message is required by a later route or filter using the same definition, the previously parsed content will be used.
- If the definition differs, the message will be re-parsed with that definition.

Some filters specify the message definition to be used when processing a message. In such cases, the message will use the filter's definition (requiring a re-parse if it differs from the previous parsed definition).

Accessing Fields

Once a message has been parsed, it is possible to access individual fields in the message by specifying the correct path. The engine allows fields to be accessed using a Segment.Field.Sub-field notation; for example, the PatientID field in the PID segment would be accessed as PID.PatientID.IDNumber.

Many components provide a field picker for selecting a field path. It is necessary, however, to explicitly specify the field path when working with the JavaScript filter. EDI Explorer can provide assistance in determining a field path from the **Message Structure** window by right clicking a field and selecting **Copy Path**. This may then be pasted into the relevant code. This functionality is useful with complex structures.

Lesson 2 of 11

Message Definitions

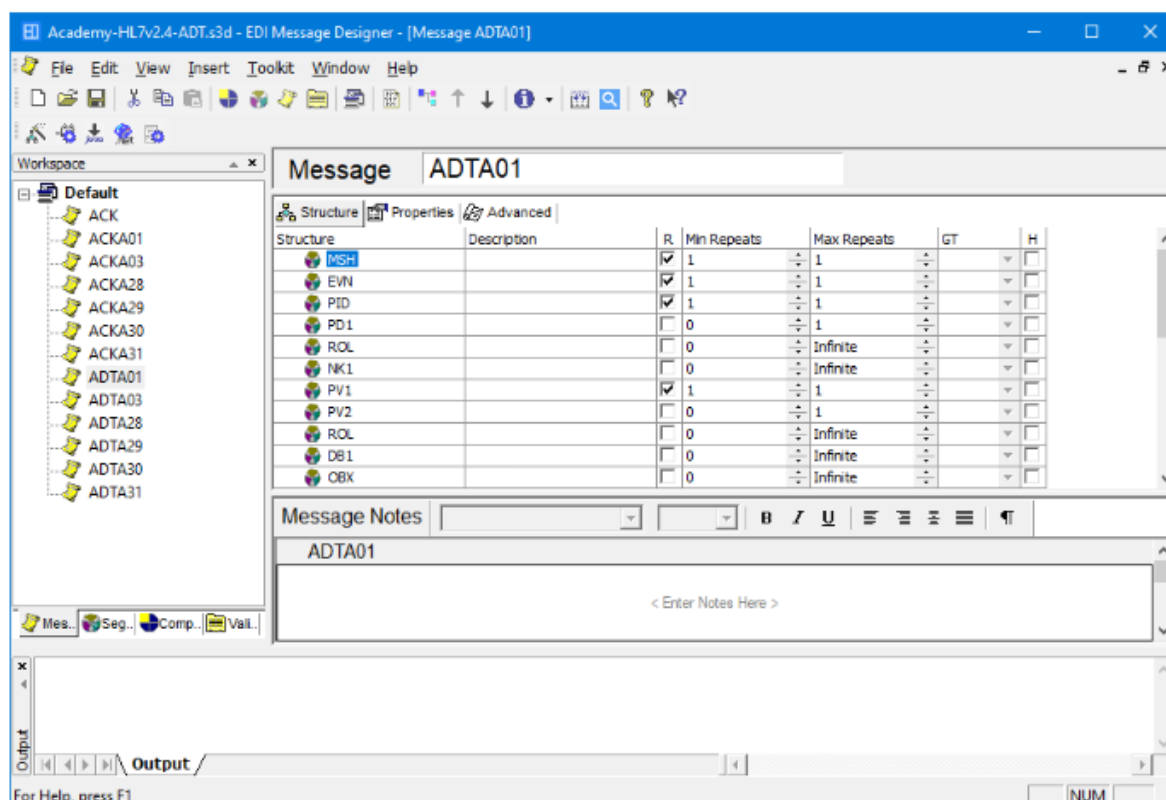
For message types except XML and JSON, Rhapsody uses a message definition created with the EDI Message Designer that is stored in a definition file (.s3d and its companion parser control file .pcf). When you create a definition, the filename should include the system that the EDI definition represents and the type of messages that can be interpreted. For example: *SimplePAS ADT HL724.s3d*.

An .s3d file contains the definitions for one or more message sets, each of which contains the definitions for one or more messages. In the most frequently used cases, each .s3d contains the definitions for one message set and multiple message types. Each message may contain multiple elements (segments) and each segment may contain multiple fields. In addition, each message set includes support for validation tables which may be used to ensure that field contains a value from a predefined set of possible values. Each message set defines a component hierarchy containing:

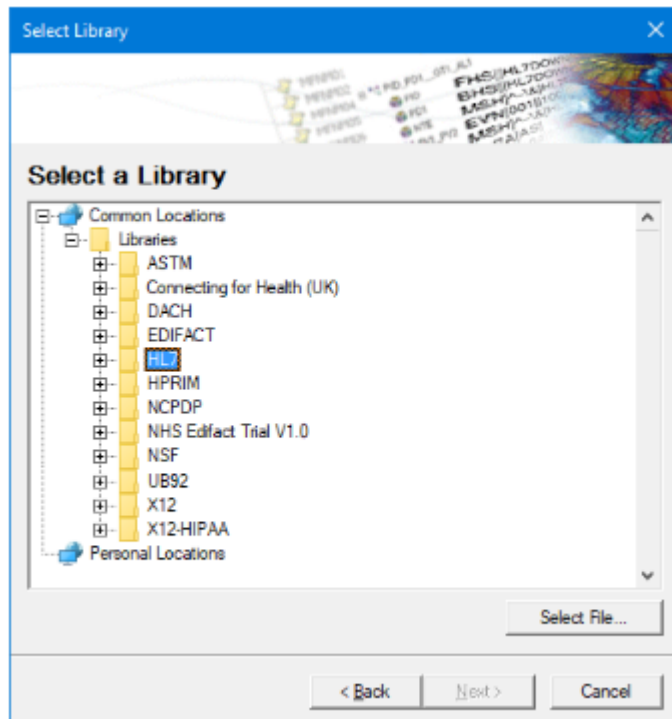
1. Message Set Properties
Describe how the message elements are defined.
2. Message Definitions
Describe the structure and components (segments) of a message.
3. Segments
Made up of groups of fields.
4. Fields
Contain individual items of information.

The definition structure supports optional and required components, grouping and repeating elements, and strict control on data types and sizing if required.

The example below shows the **Default** message set containing the ACK and ADT Message Types in an HL7 v2.4 message definition, with the ADTA01 message type highlighted in the Workspace. The **Message Structure** tab lists the segments in messages of this type, with the **MSH** (Message Header) segment highlighted. The **R** (Required) column identifies the segments which must be present in an ADTA01 message.



The EDI Message Designer ships with a library of pre-defined message definitions. These can be selected and loaded when creating a new message definition for use in Rhapsody. From the **Toolkit** menu, select **Launch Library Wizard**. When creating a definition for a standard message type, the library wizard is normally used to create the initial definition.



The library includes definitions for messages of the following types:

- **ASTM**
Used when transmitting electronic data between clinical laboratory instruments and computer systems.
- **DACH**
Used for messages containing Adult and Community Health-related information (US).
- **EDIFACT**
Used for the exchange of electronic messages between different organizations (UN standard).
- **NCPDP**
Used when mapping messages between electronic prescribing and HL7 systems.
- **NHS**
Used for exchanging medical information in the National Health System (UK).
- **UB92**
Used for Medicare billing messages (US).
- **X12-HIPAA**
Used for transmitting healthcare-related messages that meet the Health Insurance Portability and Accountability Act standard (US).

Message Set Structure

The **Message Set** structure defines rules which apply to all messages contained in the set and which are used to identify each of the components. EDI Message Designer permits the rules to be statically defined or to be derived from the message content, as is the case for HL7 messages.

The structure of the HL7 V2 message provides delimiters to parse a message into:

- Segments, which contain
 - Fields, which may contain
 - Sub-fields, which may contain
 - Sub-sub-fields, and so on
- Repeats

These delimiters are configured in the message set and can also reference the location in the message where they are defined.

Default Message Set

The **Separators** tab for an HL7 message set is shown below:

Message Set

Default

Separators

Properties

Error Settings

Advanced

☐ Initialise separators from parent message set

	Value	Replacement	Type	Usage	Field Path	Pos
Segment	\x0D	\x0D\	Fixed	Termin...		0
Repeat	~	\R\	Variable	Separator	MSH.EncodingCharacters	1
NULL	**		Fixed	Separator		0
Decimal			Variable	Separator		0
Filler			Variable	Separator		0
Field1		\F\	Variable	Initiator	MSH.FieldSeparator	0
Field2	^	\S\	Variable	Separator	MSH.EncodingCharacters	0
Field3	&	\T\	Variable	Separator	MSH.EncodingCharacters	3
Field4			Variable	Separator		0
Field5			Variable	Separator		0
User1	\	\E\	Variable	Separator	MSH.EncodingCharacters	2
User2			Variable	Separator		0
User3			Variable	Separator		0
User4			Variable	Separator		0
User5			Variable	Separator		0
Message			Variable	Separator		0
Group			Variable	Separator		0

The HL7 specification defines the location and purpose of the separators within the **MSH** segment. The **MSH** segment is required for all HL7 messages and is always the first segment in the message.

Sequence	Length	Data Type	Element Name
1	1	String	Field Separator
1	4	String	Encoding

The default segment, field separator, and encoding characters used in the message set are:

- Segments are terminated by the \x0D character (hexadecimal 0D or Carriage Return). This is a fixed value and defines the **Terminator** (end value) for the segment.
- Fields are separated by the | (pipe) character. This is a variable value that is read from the message and initiates the field. It is defined in the MSH.FieldSeparator field. Note that there is no field terminator, so a field at the end of the segment does not require a closing marker.
- Sub-fields (components in HL7 language, **Field2** in the definition) are denoted by the ^ character. In a message, this field is defined in **MSH.EncodingCharacters**, position 0.
- The Repeat character is ~. In a message, this field is defined in MSH.EncodingCharacters, position 1.
- The Escape character (used in the replacement of other encoding characters in the message) is \. In a message, this field is defined in MSH.EncodingCharacters, position 2.
- The Sub-component separator is &. In a message, this field is defined in MSH.EncodingCharacters, position 3.

When constructing an HL7 message using these defaults, we get the characteristic MSH|^~\& sequence at the start of the message.

Message set properties

The message definition supports automatic typing of messages based on the content of specified message fields. In an HL7 message, this information is in the MSH.MessageType field. When a message is parsed, the **Message Type** fields are extracted and used to match **Message Tags** defined for each message. When a match is found, the message is identified with the message name and several message properties are populated:

- **MessageName**

- **InputMessageType**
- **OutputMessageType**

This is a placeholder and may be updated as the message is processed on the route.

The definition provides the mechanism for a more fine-grained approach to identifying the message by means of the **Message Match List**.

Message Set	
Default	
Separators Properties Error Settings Advanced	
Additional Properties: ☐ Fixed Width ☒ Ignore unexpected field repeats or sub fields ☒ Trim Segments ☒ Include separators in length validation	
Message Type Fields (matched to message tags)	
Field Path	
1 MSH.MessageType.MessageType	...
2 MSH.MessageType.TriggerEvent	...

Message segments

Each message in the message set is composed of several segments, typically in a defined order. Segments may be grouped into a higher-level structure (for example, the **Procedures Group** in this example) where groupings of segments may be repeated as required by the data.

Structure Tab

The structure of a message is provided by:

- **Structure**
The name of the segment (in an HL7 message, this is the segment tag).
- **Description**
An optional description of this field in the message structure.
- **R**
Required flag. Sets **Min Repeats** to 1.
- **Min Repeats**
Minimum number of times the segment can appear in a message (0 for optional segments, 1 for required segments).
- **Max Repeats**
Maximum number of times the segment can appear in the message (1 for non-repeating segments).

- **GT (Group Type)**

This field groups several child fields. The options are

- **Ordered**

The segments must be present and in the specified order.

- **Unordered**

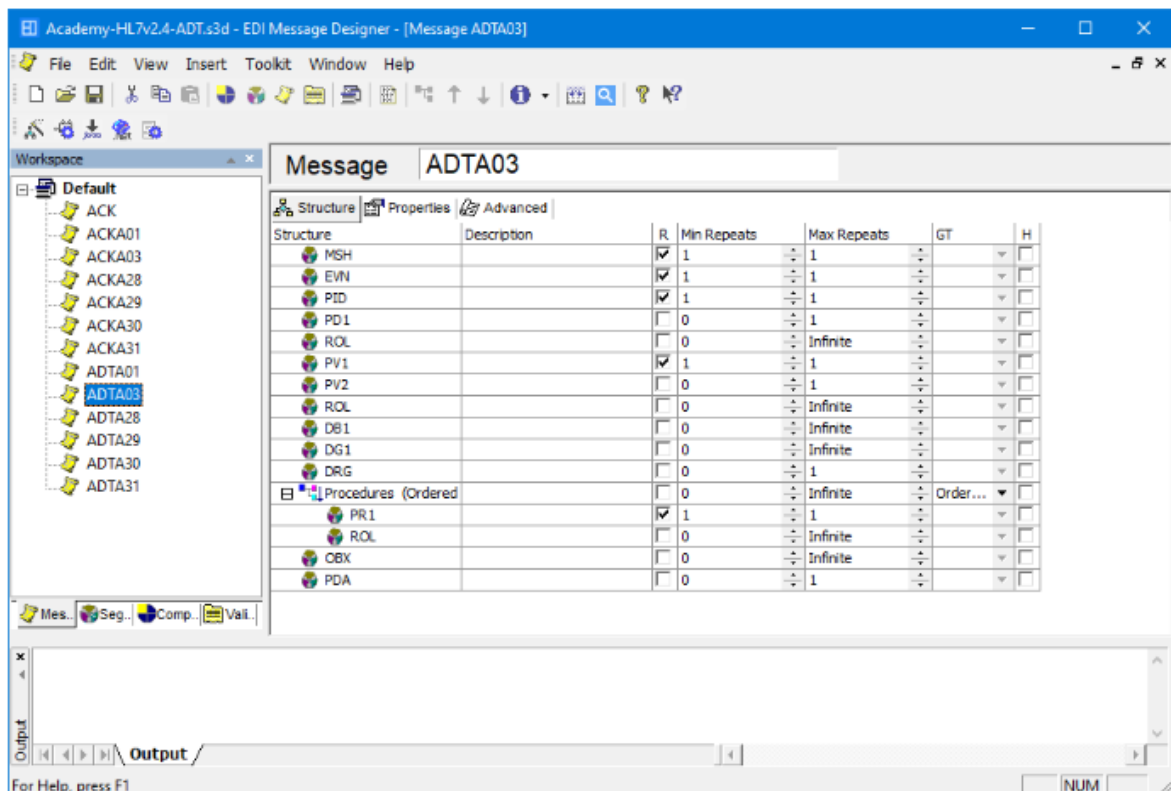
The segments must be present but may be in any order.

- **Choice**

Only one of the segments in the group may be present.

- **H (Hidden)**

Indicates that this field, while part of the message, will not be used.



Properties Tab

On the **Properties** tab of the **Segment** view you can define the segment tag, which allows the definition to provide an addressable path to the fields in the segment (in this example, the field paths would begin with **MSH**). If the segment does not contain a tag, the field **Tag is Empty** should be set.

The **Separator** is used to extract the fields from the segment and is set to **Auto** by default (that is, to use the value set for the parent message or message set). It can also be defined to use one of the **Field** or **User** values of the **Message Set** separator

definition or several message-specific values. For HL7 messages, this is normally left as the default.

The screenshot shows a configuration window for the 'MSH' segment. It has three tabs: 'Structure', 'Properties', and 'Advanced'. The 'Advanced' tab is selected. It contains the following fields:

- Segment Tag: MSH
- Tag is Empty: ☐
- Separator: Auto
- Fixed Width: ☐

Below these fields is a section titled 'Segment Match-List' with a table:

Field Path	Regular Expression to Match
...	

Advanced Tab

The **Advanced** tab contains the rarely used syntax rules, and user data for the segment.

Segment fields

Fields are defined under the **Structure** tab of the **Segment** view. In the case of HL7 segments, the segment tag is ignored and fields start at character position 4 in the segment.

The screenshot shows the 'EDI Message Designer' window with the 'Segment MSH' view. The 'Structure' tab is selected, displaying a table of fields for the MSH segment. The table has columns: Field Name, H, Datatype, Alignment, R, Min, Max, Min Rpt, and Max Rpt.

Field Name	H	Datatype	Alignment	R	Min	Max	Min Rpt	Max Rpt
1 FieldSeparator	☐	ST	Left	☒	0	1	1	1
2 EncodingCharacters	☐	ST	Left	☒	0	4	1	1
3 SendingApplication	☐	HD	Left	☐	0	180	0	1
4 SendingFacility	☐	HD	Left	☐	0	180	0	1
5 ReceivingApplication	☐	HD	Left	☐	0	180	0	1
6 ReceivingFacility	☐	HD	Left	☐	0	180	0	1
7 DateOfMessage	☐	TS	Left	☒	0	25	0	1
8 Security	☐	ST	Left	☐	0	40	0	1
9 MessageType *	☐	MSG	Left	☒	0	13	1	1
10 MessageControlID	☐	ST	Left	☒	0	20	1	1
11 ProcessingID	☐	PT	Left	☒	0	3	1	1
12 VersionID	☐	VID	Left	☒	0	60	1	1
13 SequenceNumber	☐	NM	Left	☐	0	15	0	1
14 ContinuationPointer	☐	ST	Left	☐	0	180	0	1
15 AcknowledgementType	☐	ID	Left	☐	0	2	0	1
16 ApplicationAcknowledgement	☐	ID	Left	☐	0	2	0	1
17 CountryCode	☐	ID	Left	☐	0	3	0	1

Each field is defined as follows:

- **Field Name**
The name of the field. Only the characters A-Z, a-z, 0-9 and _ (underscore) may be used.
- **H (Hidden)**
Hidden fields are typically used to hide unused fields or to force the field to use default data.
- **Datatype**
Rhapsody Messaging Toolkit defines several data types including those defined by the HL7 specification. Some fields are composite structures which are groupings of fields defined under the Composites tab of the workspace. The data type is either a primitive data type or a composite; note that the toolkit permits the definition of additional primitive types if required.
- **Alignment**
Sets the alignment of data in this field to left or right, with optional padding choices.
- **R (Required)**
Data must be present in this field for the message to be valid.
- **Min**
Minimum length for the field value.
- **Max**
Maximum length for the field value.
- **MinRpt**
Minimum number of repeats (1 for required fields, 0 for optional fields).
- **MaxRpt**
Maximum number of repeats (greater than one for for repeating fields).
- **Default**
Data entered into this field can have the following values:
 - **Counting field**
Contains the count of some items in the message.
 - **Numeric formatted string**
If the column starts with the '@' character, the information following is used as a numeric format string for number fields or a regular expression.

- **Binary field length**
If the field is a raw binary field, then this column indicates the location of the field that contains the length of the binary data.
- **Default data**
If the value in this field does not match any of the above, then the value will be used as static default data and will be inserted into the field at runtime.
- **Validation Table** (not shown in the screenshot)
Validation tables are defined on the workspace **Validation** tab and are used to ensure that data in the corresponding field conforms to the expected values.
- *Lesson 3 of 11*

• **Creating an HL7 Message Definition**

The Rhapsody Messaging Toolkit tool set includes a library of definitions which covers the common standards used in the Healthcare industry. These definitions conform to the standards document and are therefore likely to need minor modification to accommodate system-specific customizations. However, they provide a mechanism to rapidly create the base definition set from which to proceed. The following example illustrates the creation of an HL7 message definition using the **Library Wizard**, accessed from the EDI Message Designer.

Select library

1. 1

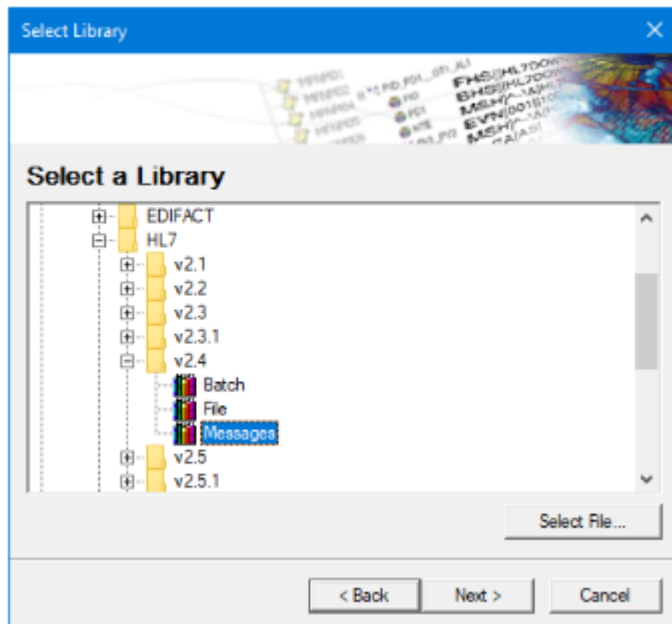
Open EDI Message Designer and, from the **Toolkit** menu, select **Launch Library Wizard**.

2. 2

This first screen displays a '**Welcome to the Wizard**' message; select **Next** to continue.

3. 3

The second screen lists the message libraries; select **HL7 > v2.4 > Messages**.

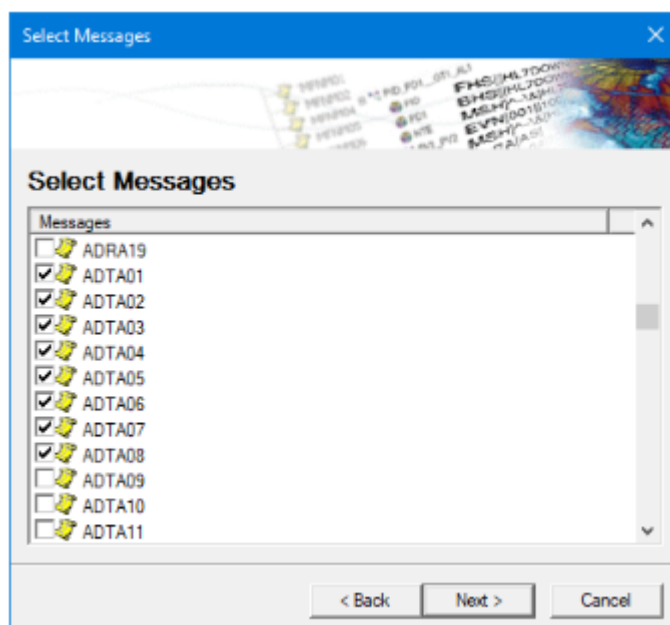


Generally, messages are sent singly rather than in a batched form. The **HL7 Batch** format provides batch header and trailer markers for HL7 messages, while the **File** format provides file header and trailer markers for a batch of HL7 messages.

Select messages

1. 4

Select **Next** to select the message types to be included in the message definition. For this example, the ACK message type has been selected (not shown in the following screenshot) followed by the ADTA01 - ADTA08 message types.



1. Multiple messages can be selected at the same time by using **Ctrl+click** (for a discrete selection) or **Shift+click** (for a continuous selection), then selecting the checkbox associated with any of the selected rows.

Load messages

1. 6

Select **Next** followed by the **Finish** button to complete the wizard and load the selected message set into EDI Message Designer.

Lesson 4 of 11

Creating an Ad-hoc Message Definition

The following example illustrates the creation of a message definition for a comma separated file. This file has the following characteristics:

- The first line of the file is a header record.
- Each segment is terminated by a new line.
- Each field is quoted (for example, "<field_content>").
- Each field is separated by a comma (,).
- There is no message type information in the file.
- There are no segment tags.

```
PatientId,StreetAddress,Suburb,City,PostCode,Country  
84568-4564,1 Molesworth Street,Pipitea,Wellington,6011,NZL  
84502-7664,99 Khyber Pass Road,Grafton,Auckland,1023,NZL  
87619-9902,211 Oxford Terrace,Christchurch Central,Christchurch,8011,NZL  
84568-4341,6 Ohakune Mountain Road,,Ohakune,4625,NZL  
84568-3168,221 Kent Street,,Kingston,9793,NZL
```

Configuration Approach

The approach for creating ad-hoc message definitions is generally as follows:

1. 1

Configure the **Message Set** with the details of the message structure.

2. 2

Create the **Messages** for this definition.

3. 3

Create the **Segments**, **Composite** structures and **Validation Tables**.

4. 4

Add **Segments** to **Messages**.

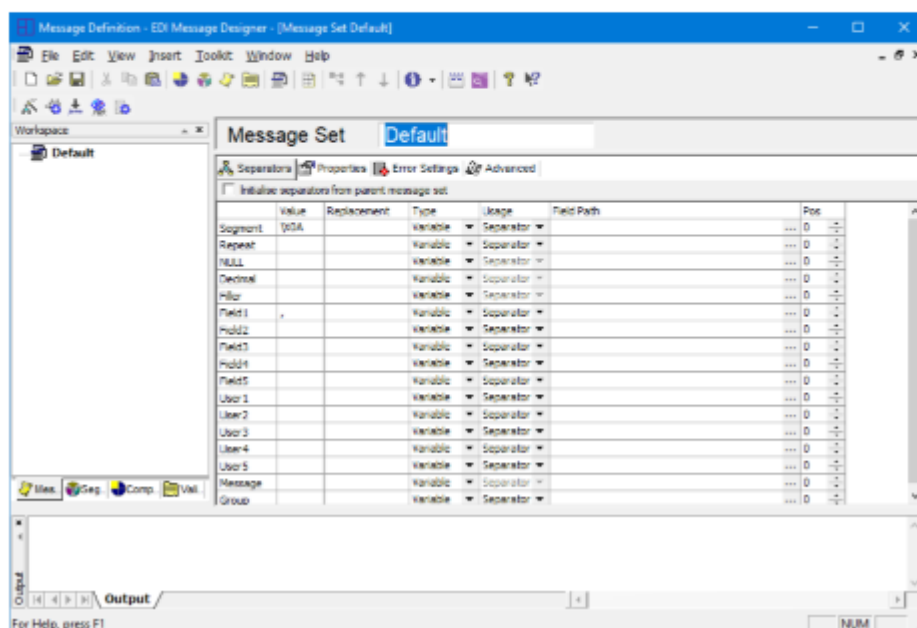
Configure the message set

The page is only an outline of the ad-hoc message creation process. A later exercise will show the creation of CSV Messages in more detail.

1. 4

Double-click the **Default** icon in the Message Designer workspace to access the properties of the default message set. For CSV files, the following are normally used:

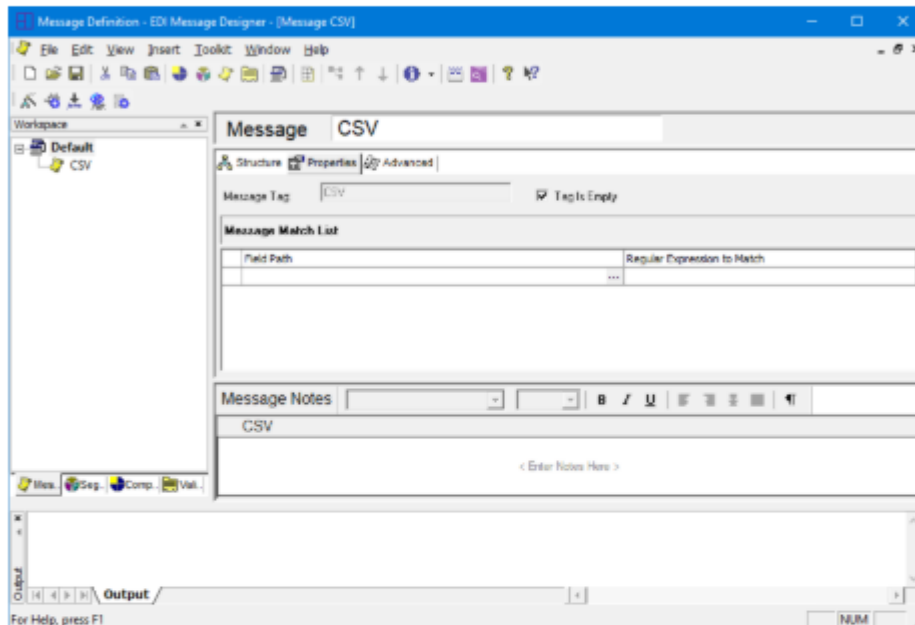
Segment	The marker which separates segments; in this case, the new-line character. Specify this in hexadecimal notation (\x0A).
Field1	The Field 1 separator is used to separate fields in the message (Fields 2 - 4 are used for the sub-field hierarchy). CSV files use the ',' (comma) as the separator.



Create a new message

5

Select the **Messages** tab below the **Message Designer** workspace. From its context menu, select **New Message**. Provide a descriptive name for the message, for example CSV, and on the **Properties** tab on the right, ensure that the **Tag is Empty** checkbox is selected.



6

The **Message Notes** panel can be populated with explanatory information relating to the message definition being created. If this panel is not visible, bring it to view by selecting the **Notes Pane** entry under the **View** menu.

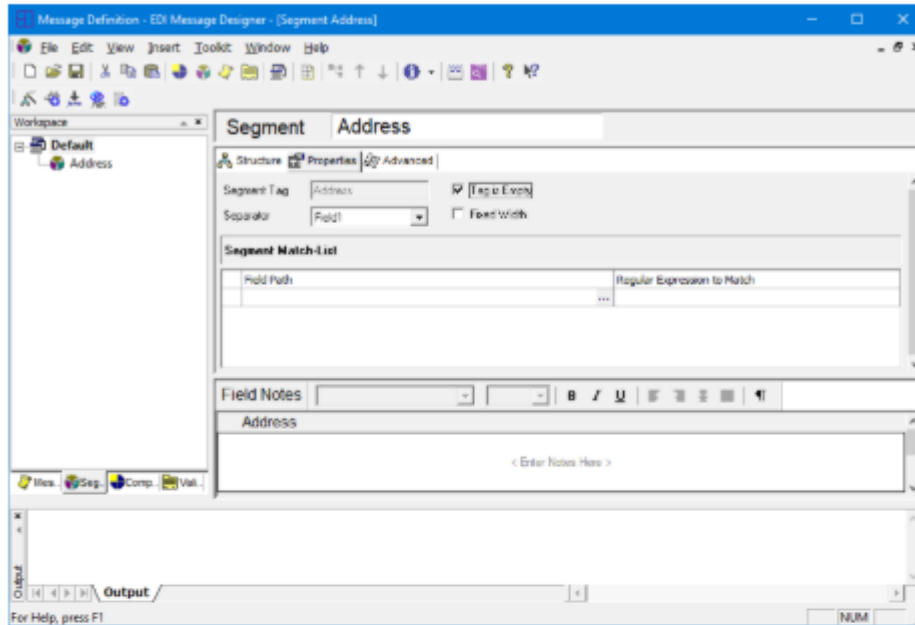
Create a message segment

7

Select the **Segments** tab below the **Message Designer** workspace. From its context menu, select **New Segment**. Name the new segment, for example Address and, on the **Properties** tab, ensure that the **Tag is Empty** checkbox is selected and the **Fixed Width** checkbox is not selected.

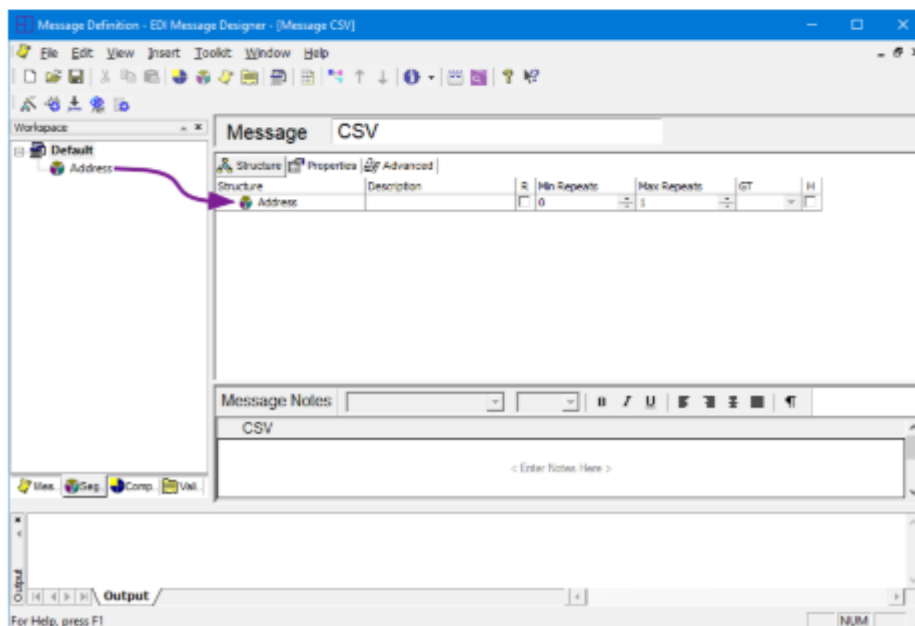
Create a message segment

- 7 Select the **Segments** tab below the **Message Designer** workspace. From its context menu, select **New Segment**. Name the new segment, for example **Address** and, on the **Properties** tab, ensure that the **Tag is Empty** checkbox is selected and the **Fixed Width** checkbox is not selected.



Add the segment to the message

- 8 Segments are added to a message by dragging them from the **Segments** workspace into the message's **Structure** tab, as shown below.



Validating Messages and Definitions

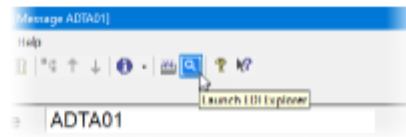
Lesson 5 of 11

Validating Messages and Definitions

A message definition can be validated prior to its use with the Rhapsody engine by loading it into EDI Explorer and checking it against one or more test messages. This process highlights issues with the definition, such as where a field in the message is longer than specified or the content does not conform to validation rules. This process can also be used to test messages against an existing message definition.

Launching EDI Explorer

EDI Explorer is launched by selecting its icon from the Message Designer's toolbar (you will be prompted to save the definition if you have not already done so). This opens the application and automatically loads the current message definition. If EDI Explorer is opened without first referencing a message definition (for example, when opened directly from Windows), the message definition will need to be opened from the **File** menu before any test messages are added.



Loading Test Messages

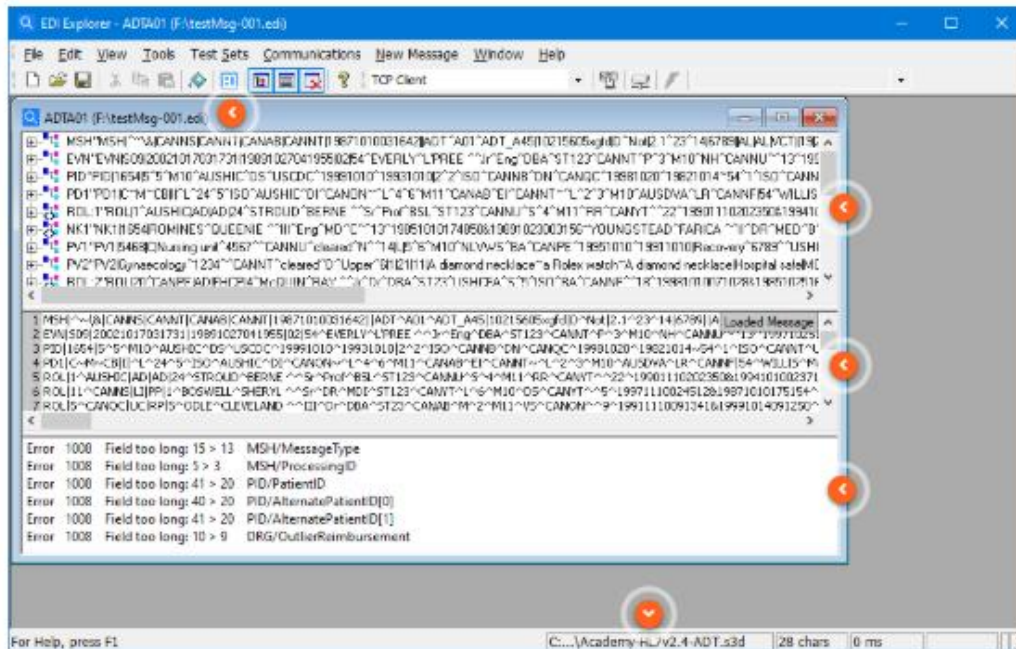
Test messages are loaded either by selecting the **File** menu, by selecting **Import Flatwire Message** from **File** or by dragging and dropping the files onto the application.

Viewing a Test Message in EDI Explorer

The most common message format is a plain text file (often described as a flatwire file due to its pipe-delimited format); see the following screenshot for an example.

Viewing a Test Message in EDI Explorer

The most common message format is a plain text file (often described as a flatwire file due to its pipe-delimited format); see the following screenshot for an example.



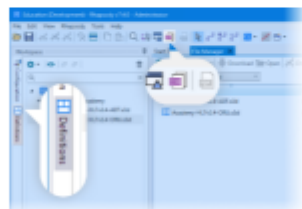
From EDI Explorer you can either save messages in the EDI Explorer specific .sem format, or you can use the **File > Export Flatwire Message To File** option to export the message in a format that can be used by other applications.

Lesson 6 of 11

Managing Definition Files

Many routes, communication points and filters use a Message Definition file as part of their configuration. The Mapper filter requires a Mapper Definition file in its configuration. Other components may require an XML Schema Definition file in their configuration.

These files are added and managed in Rhapsody in either the **File Manager** or on the **Definitions** tab in the IDE workspace, both of which are shown in the following image.

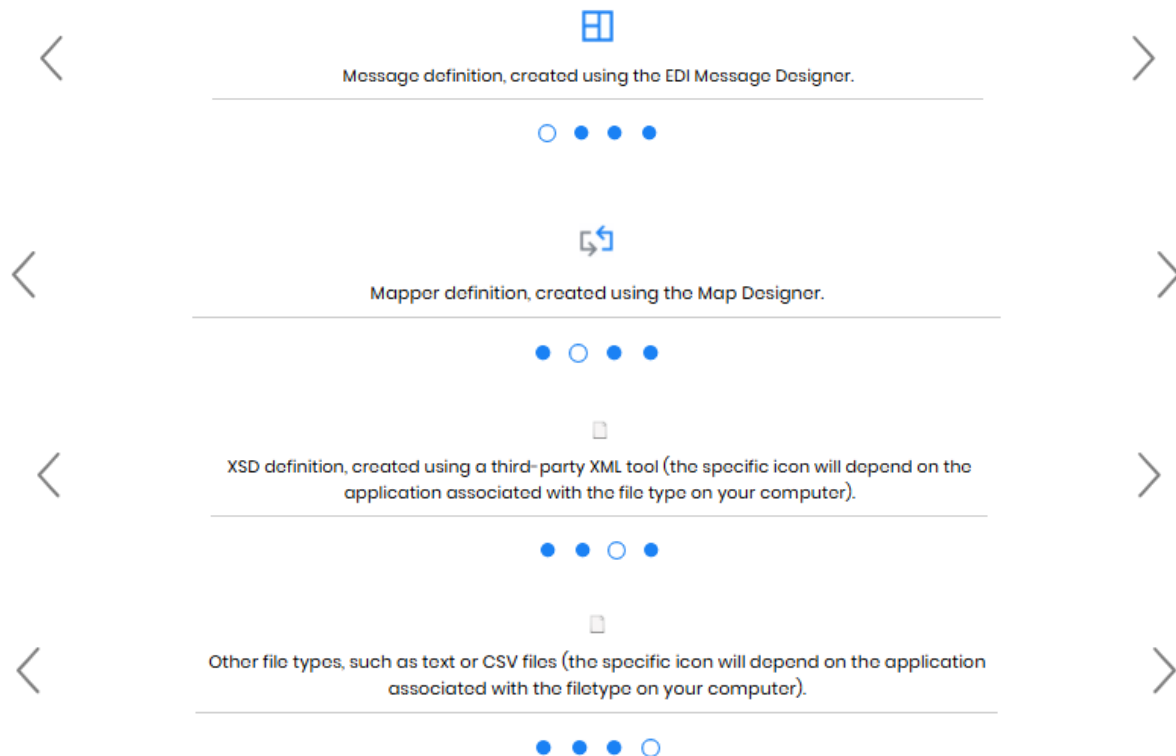


The File Manager

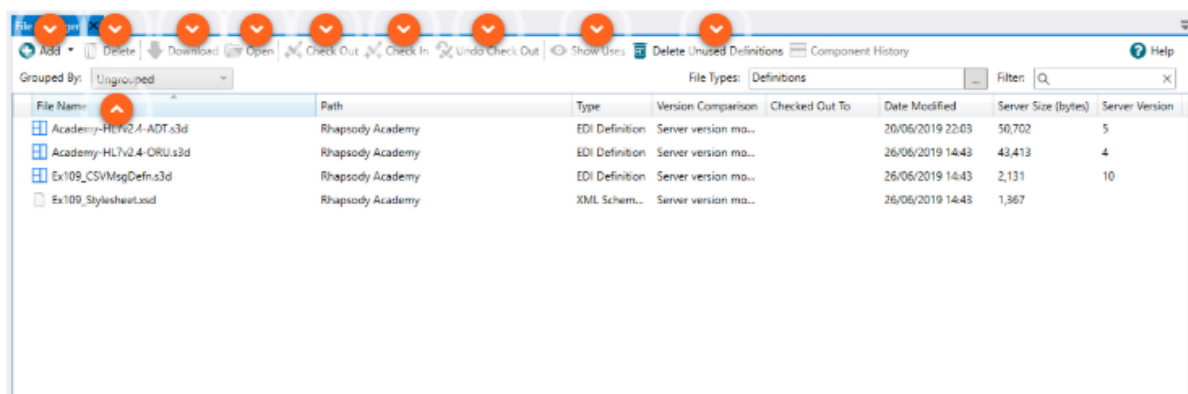
The **File Manager** is opened in a new tab by selecting its icon in the IDE toolbar.

Message and Mapper definitions are listed in the manager by default. If there are other file types available, they can be included in the display by selecting the **Definitions and Files** entry in the **View** menu.

A file's type is identified by its accompanying icon. The options include:



The buttons at the top of the manager provide access to the following functionality. Functions that cannot be used with the currently selected file are grayed out.



If you can't see a definition in the File Manager that you expect to see, check the File Types and Filter options to make sure it hasn't been filtered from view.

Version comparison

The **Version Comparison** column compares the version of the definition or configuration file stored locally in the Rhapsody IDE on your machine with that on the Rhapsody engine (server). The options are:

- **Not Downloaded**

There is no local copy of the definition. Select **Download** above to download a copy to the current machine where it can be checked out before making changes to its content.

This occurs when the file was created on another machine, possibly, but not necessarily by another administrator.

- **New**

There is no engine copy of the definition. Select **Upload** to send a copy to the Rhapsody engine.

This occurs when the **Add** option has been used to add a new definition to the File Manager.

- **Same as server**

The version of the locally stored definition and the version on the engine is the same. When this is the case, the **Status** column will:

- Be blank, which means that the file is currently checked in.
- Show *Checked out*, which means that the local version could potentially be different from the one on the engine.
- Show *Checked out by* which means that the definition is currently checked out to the named user. In such a case, the options are:
 - Open the definition in read-only format. If changes are required, they can only be saved by providing a new filename.
 - Select **Undo Check Out** to discard the previous administrator's changes and open the definition in an editable format. This option is often required when the 'other administrator' is not available to log in and complete his or her work.

- **Server version modified**

The server (engine) version of the definition is a later version compared with the one stored locally. Select **Download** to overwrite the local version with the one from the server.

- **Local version modified**

The local copy of the definition is a later version compared with the one on the engine. In such a case, the options are to select:

- **Check In** to copy the local configuration to the engine.

- **Upload** to copy the local configuration to the engine, leaving its status as Checked Out.
- **Undo Check Out** to overwrite the local copy with the version downloaded from the engine.
- **Download** to overwrite the local copy with the version downloaded from the engine, leaving its status as Checked Out.

The **Date Modified** column displays the date of the last status change for the corresponding file. The **Server Size** column lists the space occupied by the file, in kilobytes (KB).

The Definitions Tab

The **Definitions** tab is displayed in the workspace; selecting it displays a list of the message and mapper definitions loaded onto the Rhapsody engine.

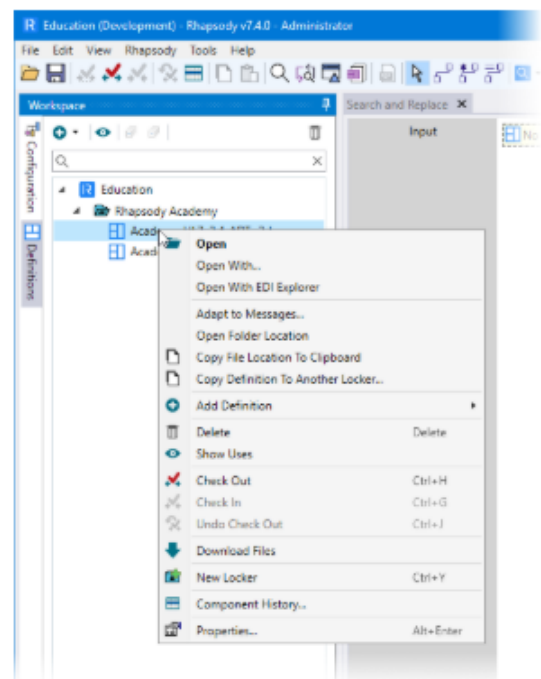
To access the **Definitions** tab, select the **Definitions** tab from the left of the workspace.

New or existing definitions can be added as follows:

- From the context menu in the tab, select **Add Definition** followed by **Add Existing Definition**.

If you choose **Add New Definition**, EDI Message Designer opens from where you will be able to begin creating the new definition using the methods described in this module. Navigate to the location of the existing definition and select it. The definition will be listed in the workspace with an accompanying red tick to indicate it is checked out. Check in the definition from its context menu.

- Once this has been completed, the definition is ready for use with other Rhapsody components.



Use **Shift + click** or **Ctrl + click** to select multiple definitions. Then, from the context menu, you can choose from the following actions:

- **Open/Open With**
Open the selected file either with its default application or a selected alternative.
- **Adapt to Messages**
Initiate an analysis of the selected .s3d message definition by comparing it with

one or more test messages. The definition can then be updated with any changes.

- **Open Folder Location**

Opens Windows Explorer followed by the folder in which the selected definition file is located. Alternatively, the file's location can be copied to the clipboard.

- **Copy to another Locker**

Places a copy of the selected definition into a specified locker.

- **Add Definition**

Add an existing or new definition to the Rhapsody engine.

If you choose **Add New Definition**, EDI Message Designer opens from where you will be able to begin creating the new definition using the methods described in this module. Navigate to the location of the existing definition and select it. The definition will be listed in the workspace with an accompanying red tick to indicate it is checked out; check in the definition from its context menu. Once this has been completed, the definition is ready for use with other Rhapsody components.

- **Rename/Delete Definition**

Rename or delete the selected definition.

- **Expand/Collapse Children** (only applicable if a locker is selected)

Expand or collapse the locker to hide or list its contents.

- **Show Uses**

List the routes, communication points and filters that use this definition as part of its configuration.

- **Check Out/In/Undo**

Check the selected definition out or in, or undo a previous check-out.

- **Download File**

Download the selected definition to a local or network location.

- **New Locker**

Create a new locker into which, for example, copies of existing definitions can be placed.

- **Component History**

View a history of changes made to the selected definition or roll back its configuration to an earlier version.

- **Properties**

Opens the standard Windows **Properties** dialog for the selected file. Information in this dialog should be treated as view-only.

- *Lesson 7 of 11*

• **Exercise – Build an EDI Message Definition**

In this exercise we will use the Library Wizard in the EDI Message Designer to build an HL7 message. The new definition will then be tested against some test messages in EDI Explorer.

Resources

This exercise makes use of the messages available in the *. \Associate Files and Definitions\109 – Message Definitions\HL7* folder in the resources Zip file available at the start of the course.

Creating and Editing a Message Definition

1. 1

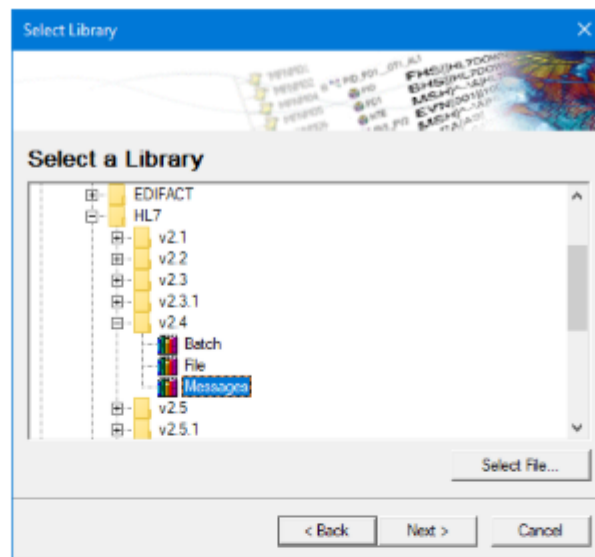
Open the EDI Message Designer by selecting the icon from the **Rhapsody Messaging Toolkit** drop-down in the IDE or from the Windows Start menu.

2. 2

From the **Toolkit** menu, select **Launch Library Wizard**, then select **Next**.

3. 3

On the **Select a Library** screen, select **HL7 > v2.4 > Messages**.



4

Select **Next**, then select the following message types:

- **ACK** (this exercise uses the generic ACK message rather than specific ones for each message type)
- **ADTA01** (admission message)
- **ADTA02** (transfer)
- **ADTA03** (discharge)
- **ADTA04** (registration)
- **ADTA05** (pre-admission)
- **ADTA06** (transfer outpatient to inpatient)
- **ADTA07** (transfer inpatient to outpatient)
- **ADTA08** (information update)

Make sure you scroll past all the ACK message definitions to find the ADT message definitions.

1. 5

Select **Next**, then select **Finish** to create the message definition containing the chosen message types.

2. 6

From the **File** menu, select **Save as** to save the message definition using the format: <username>_Exercise_HL7v2.4.

Edit an existing message definition

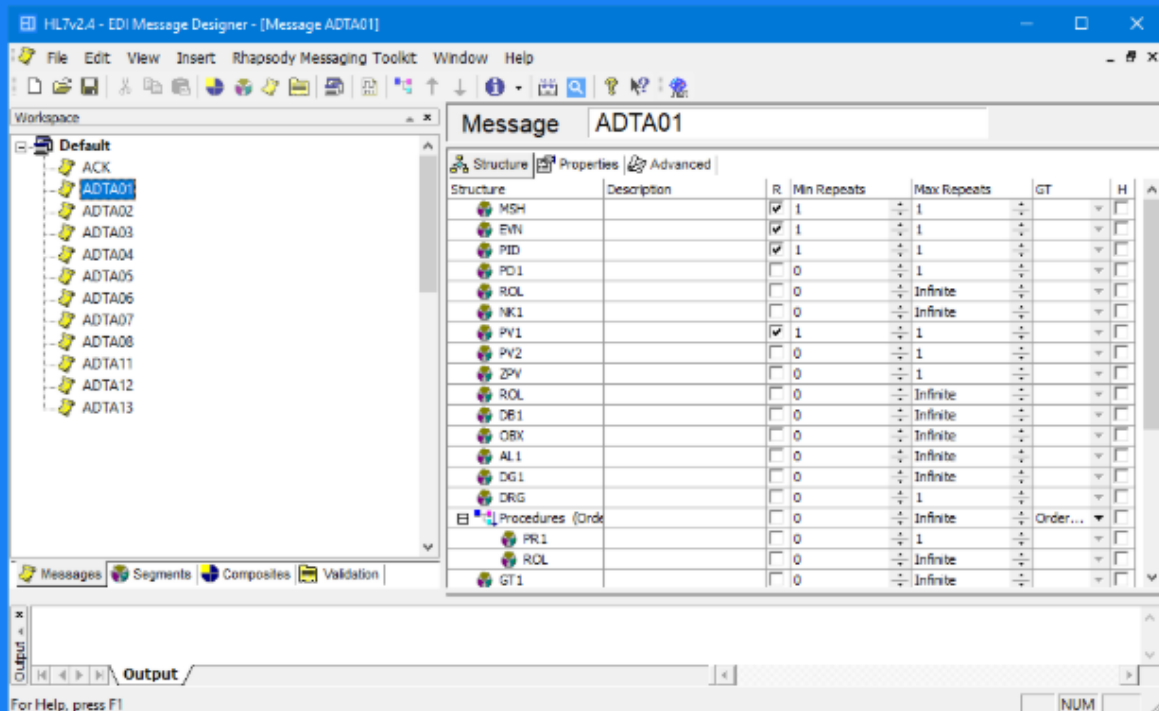
A new message type is added to a definition by selecting the **Library Wizard** followed by the messaging standard (normally HL7) and version of the existing definition. Select the message types to add and select **Next**, then **Finish** to complete the process.

1. 7

Add the following message types to your definition file, then **Save** the changes:

- ADTA11 (cancel admission)
- ADTA12 (cancel patient transfer)
- ADTA13 (cancel patient discharge)

Model Answer



Checklist

- Check that the message types listed in the **Workspace** match those selected from the library.
- Double-click the **ADTA01** message type to list its segments in the right-hand panel.
- Double-click the **MSH** segment to list its fields and composites.
 - The same list can be displayed by selecting the **Segments** tab at the base of the workspace and double-clicking the **MSH** segment.
- Expand the **MSH.SendingApplication** composite in the right-hand panel to list its associated fields.
 - The same list can be displayed by selecting the **Composites** tab at the base of the workspace and double-clicking the **HD** composite.

View the Message in EDI Explorer

Ensure that the following EDI Explorer panels are visible:

1. Message Structure (top panel)
2. Flat File representation of the test message (middle panel)

Exercise - EDI Messages

The purpose of this exercise is to configure an instance of EDI Explorer to connect with the engine.

1. Configure an instance of EDI Explorer to mimic a Laboratory System that needs to exchange EDI messages with Rhapsody.
2. Configure Rhapsody to generate an acknowledgement for every message sent to the route. Return the acknowledgement to the originating system.
3. Configure Rhapsody to validate incoming messages and, if validation errors are present, to insert a description of the error into their matching acknowledgements.

This exercise illustrates the value of EDI Explorer as a Rhapsody testing tool. It also shows how this tool can be used to edit EDI message content on the fly and to see the effect of changes as they are made.

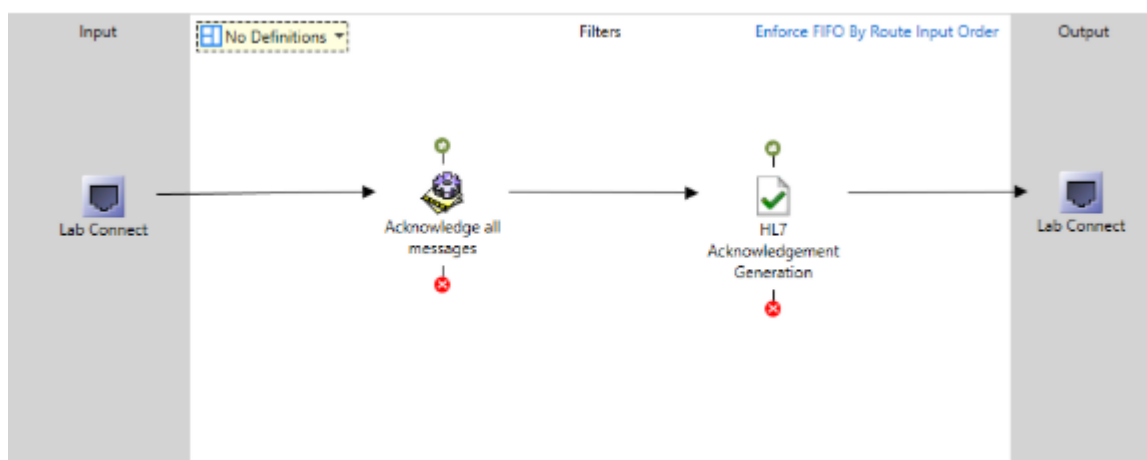
Resources

This exercise makes use of the messages and message definitions available in the `.\Associate Files and Definitions\109 - Message Definitions\HL7` folder in the resources Zip file available at the start of the course.

Construct the Route

We will start by constructing a route that receives messages from EDI Explorer which is standing in for an external laboratory system. Every message sent into the route will be validated and an acknowledgement generated. If a non-valid message is encountered, details of the error will be inserted into the acknowledgement.

Create a new route named **ACK and Validate** using the image below as a guide. We will configure the route in the following steps.

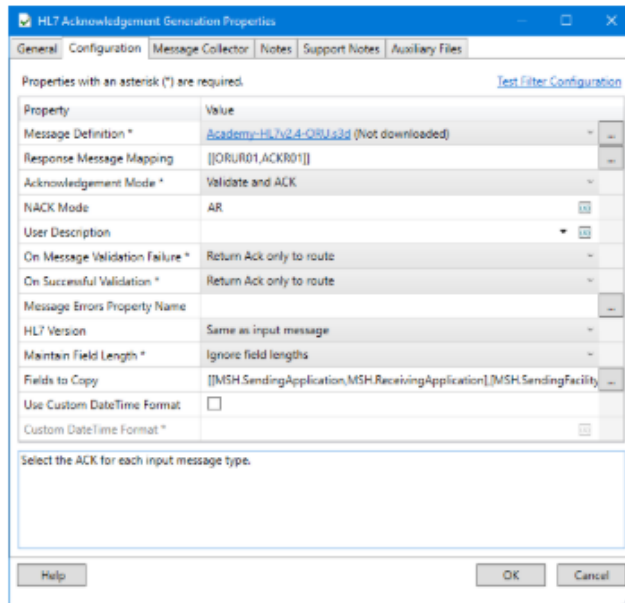


TCP server configuration

Create a bidirectional TCP Server communication point and add it to both sides of the route. Leave all properties at their default values except for **Local Port**, which can be given any value in the range 1025 - 65535. Throughout this exercise, a value of 10070 will be used.

HL7 acknowledgement generation configuration

Add an HL7 Acknowledgement Generation filter to the route and, on its **Configuration** dialog, specify the following properties:



HL7 Acknowledgment Generation Properties

General Configuration Message Collector Notes Support Notes Auxiliary Files

Properties with an asterisk (*) are required. [Test Filter Configuration](#)

Property	Value
Message Definition *	Academy-HL7v2.4-ORU.s3d (Not downloaded)
Response Message Mapping	[ORUR01,ACKR01]
Acknowledgement Mode *	Validate and ACK
NACK Mode	AR
User Description	
On Message Validation Failure *	Return Ack only to route
On Successful Validation *	Return Ack only to route
Message Errors Property Name	
HL7 Version	Same as input message
Maintain Field Length *	Ignore field lengths
Fields to Copy	[MSH.SendingApplication,MSH.ReceivingApplication ,MSH.SendingFacility]
Use Custom DateTime Format	☐
Custom DateTime Format *	

Select the ACK for each input message type.

Help OK Cancel

- **Message Definition**

Select the *Academy HL7v2.4-ORU.s3d* message definition supplied in the exercise resources.

- **Response Message Mapping**

Specify that the generated acknowledgement is to be based on an **ACKR01** message type, rather than the default **ACK**.

Both acknowledgement message types are included in the *Academy HL7v2.4-ORU.s3d* message definition.

- **Acknowledgement Mode**

Validate and ACK. This setting forces the filter to check incoming messages for valid content:

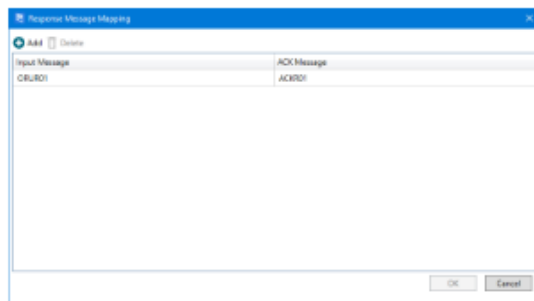
- Fields identified as required contain data.
- Fields with a specified maximum field length do not contain data that exceeds this value.
- Fields with an associated validation table do not contain data that has no match in this table.

- **On Message Validation Failure**

Return ACK only to route (no change from default)

- **On Successful Validation**

Return ACK only to route (no change from default)



Response Message Mapping

Add Remove

Input Message	ACK Message
ORUR01	ACKR01

OK Cancel

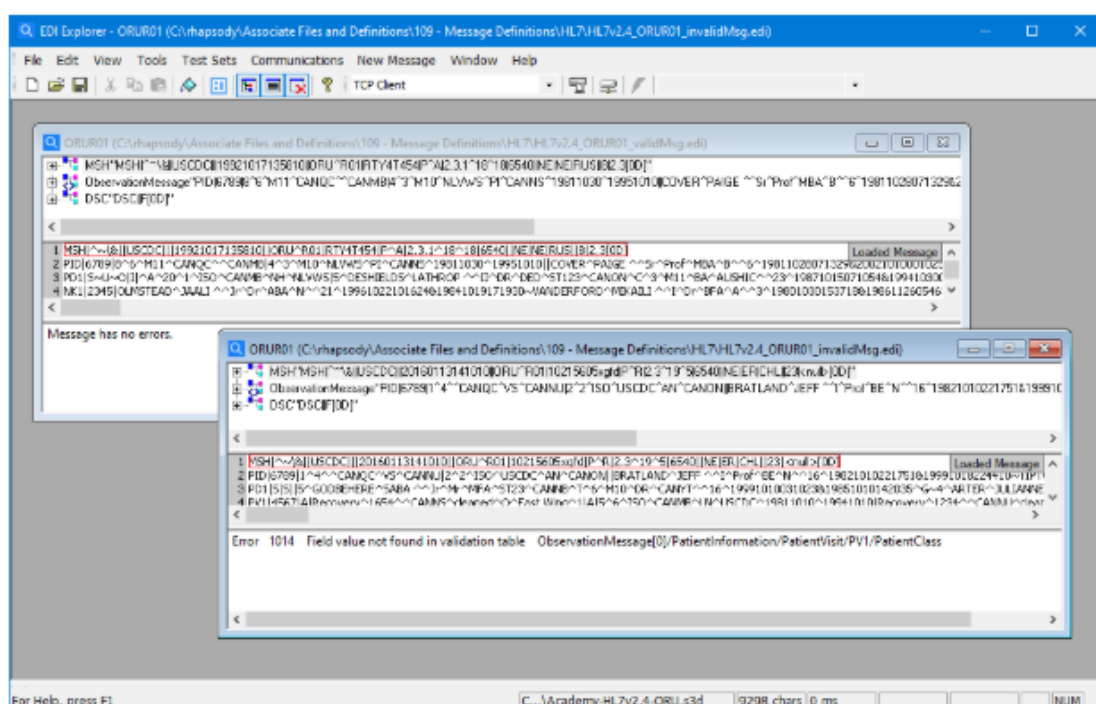
Complete the route

Add and name the No-operation filter, then join all components to match the screenshot of the route above. Check in and start the route and communication point.

Configure EDI Explorer

Open EDI Explorer and load the *Academy-HL7v2.4-ORU.s3d* message definition. Import the two test messages, *HL7v2.4_ORUR01_validMsg.edi* and *HL7v2.4_ORUR01_invalidMsg.edi*.

Once completed, your screen should be similar to that shown below:



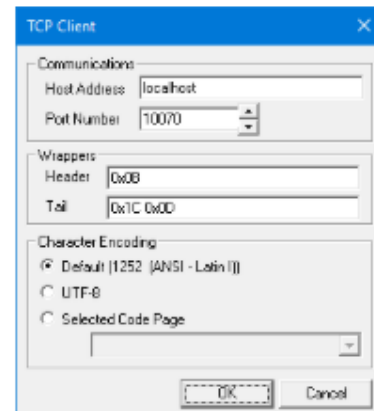
Communications setup

- From the **Connection Type** drop-down list, select **TCP Client**. This setting needs to be chosen because the communication point it will be connecting to in Rhapsody is configured as a TCP Server.

Communications setup

- From the **Connection Type** drop-down list, select **TCP Client**. This setting needs to be chosen because the communication point it will be connecting to in Rhapsody is configured as a TCP Server.

- Select **Configure the Communications Setup** in the EDI Explorer toolbar. On the resulting dialog, specify the same port number value as was used in the configuration of the TCP Server communication point in Rhapsody; in this case, 10070. Select **OK** to close the dialog, leaving all other properties at their default values.



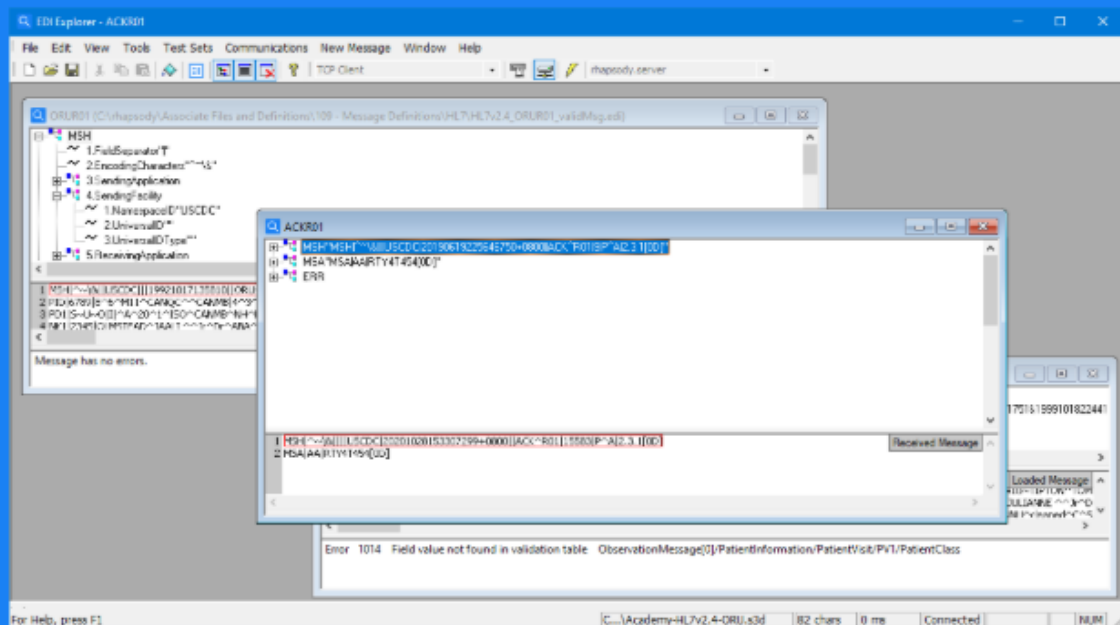
- Once the port number has been specified, select **Connect**. If EDI Explorer can find a TCP server listening on the same port number, the **Send** icon will become available.

EDI Explorer is now ready to send one or more test messages to Rhapsody.

Test with a Valid Message

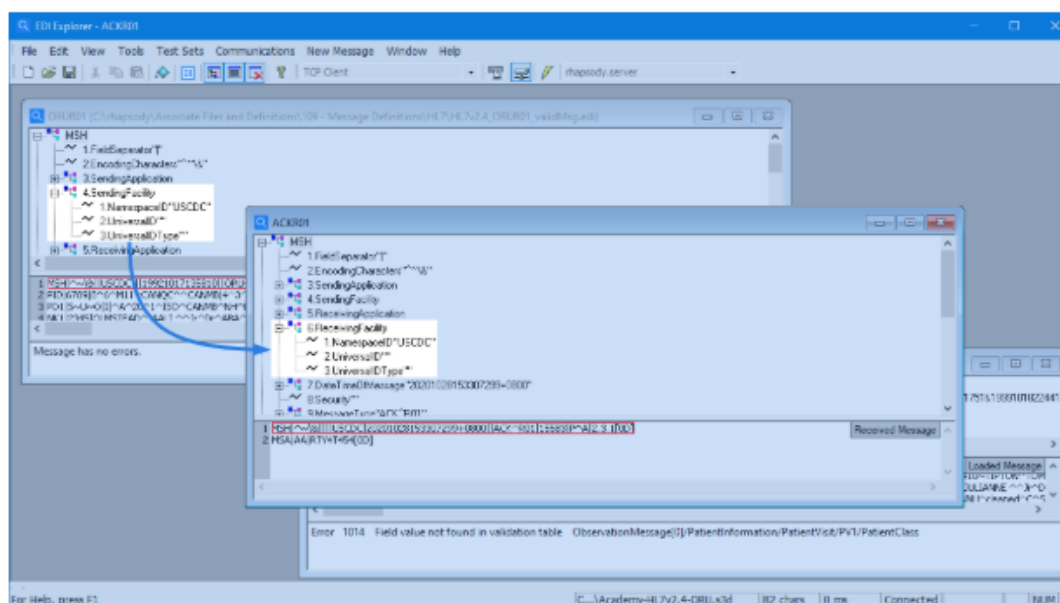
Select the *HL7v2.4_ORUR01_valid.edi* test message in EDI Explorer then select **Send**. This action sends the message into Rhapsody where it is validated, and an acknowledgement is generated. This acknowledgement passes to the output communication point, from where it is returned to EDI Explorer.

The **ACKR01** acknowledgement appears in EDI Explorer, as shown below:

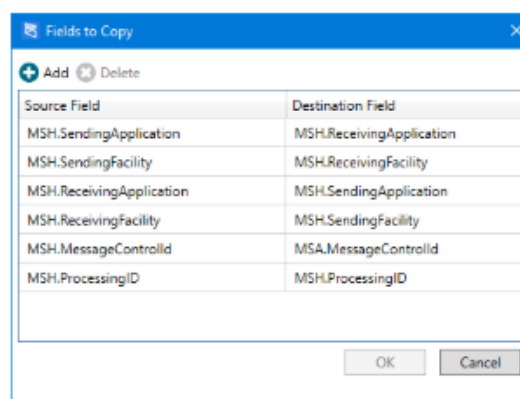


Field Mappings

Expand the MSH segment in the original message and note that the **SendingFacility.NamespaceID** field is populated with the value of 'USCDC'. Expand the same segment in the **ACKR01** acknowledgement and note that this value has been mapped to the **ReceivingFacility.NamespaceID** field.



This mapping results from the default configuration of the HL7 Acknowledgement Generation filter in Rhapsody. Open this filter's **Properties** dialog and select its **Configuration** tab. Open the dialog associated with the **Fields to Copy** property, which should be similar to this example:

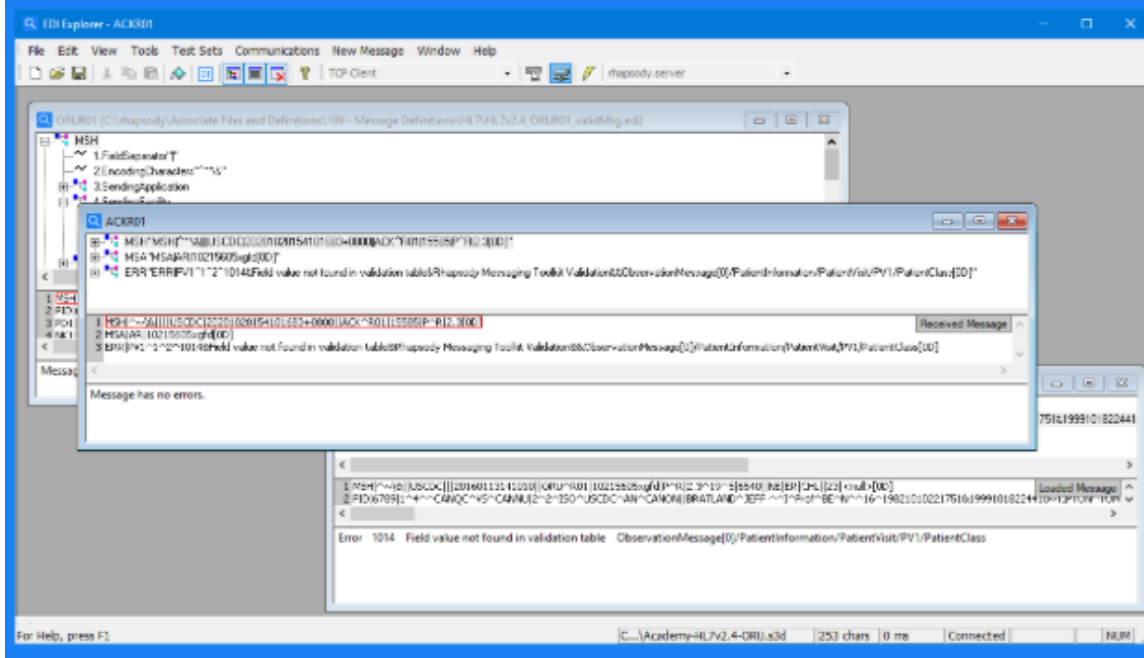


The logic is that the Center for Disease Control facility, in this case, sends the original message, and receives the acknowledgement generated from it.

Select the **HL7v2.4_ORUR01_invalid.edi** test message in EDI Explorer then select **Send**. This action sends the message into Rhapsody where it will fail validation. An acknowledgement is still generated, with a description of the failure reason inserted into its ERR segment. This acknowledgement is then passed to the output communication point, from where it is returned to EDI Explorer.

Model Answer

The **ACKR01** acknowledgement appears in EDI Explorer, as shown below. Notice that the **ERR** segment is populated with details of the valuation errors.



Error Correction

The full text of the error message resulting from the processing of the invalid test message is:

Field value not found in validation table&Symphonica

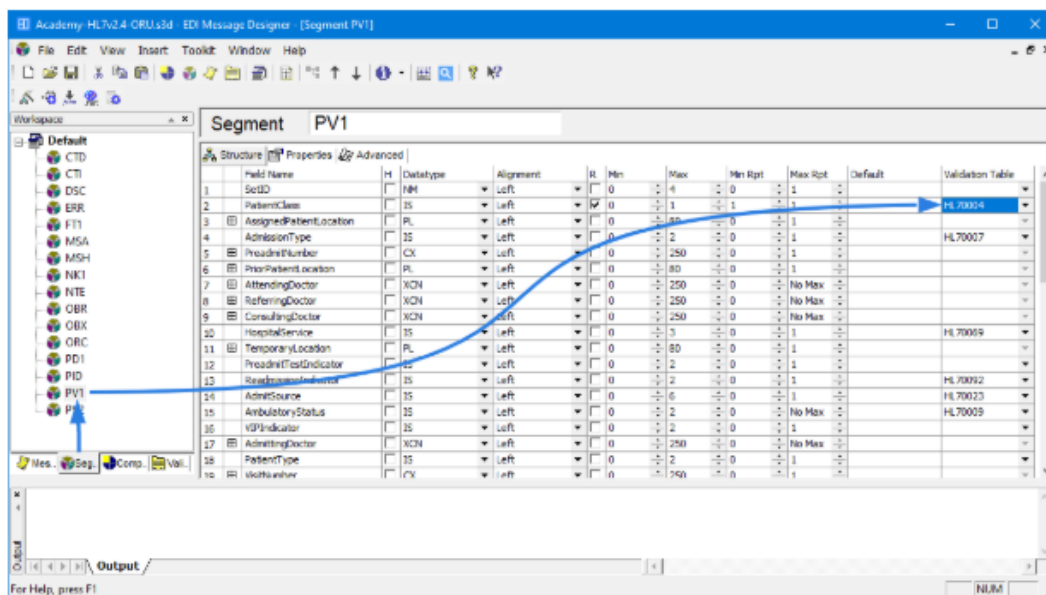
Validation&&ObservationMessage[0]/PatientInformation/PatientVisit/PV1/PatientClass[0D]

This error can be corrected with the following steps:

- 1.

Locate the field containing invalid data.

Open the *Academy-HL7v2.4_ORU.s3d* message definition in EDI Message Designer and locate the ObservationMessage.PatientInformation.PatientVisit.PV1.PatientClass field. Note that this field has an associated Validation Table: **HL70004**.

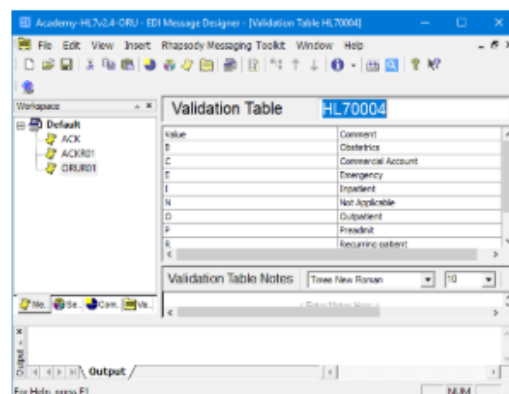


2

Open the PatientClass Field's associated Validation Table

Right-click the **HL70004** Validation Table's entry associated with the **PV1.PatientClass** field in EDI Message Designer and, from the context menu, select **Go to Validation Table Definition**. The resulting screen will be similar to this example.

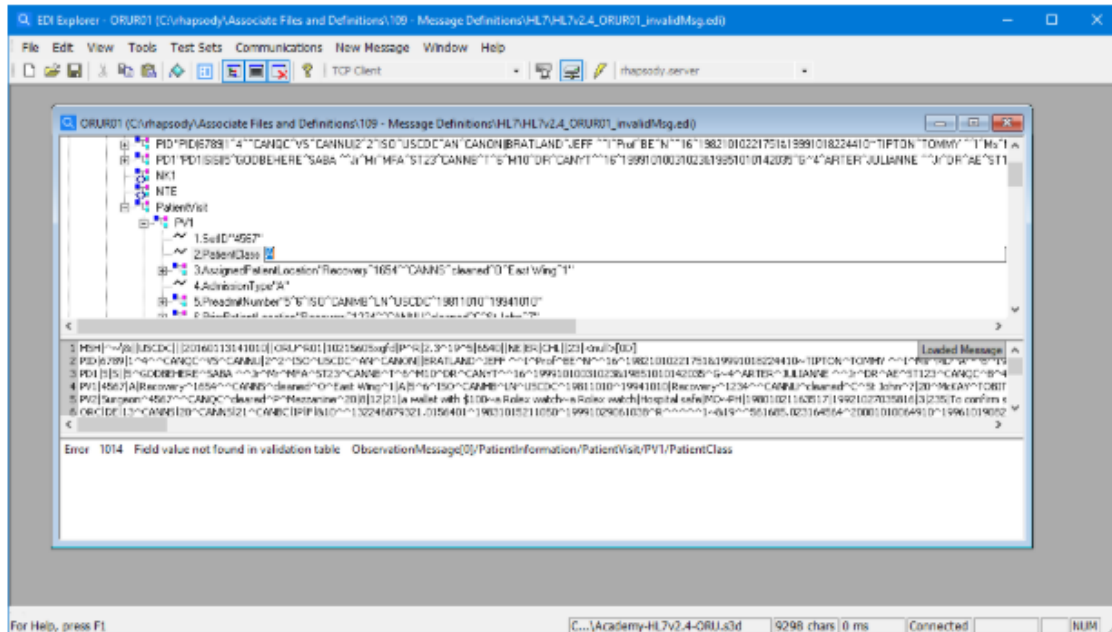
View the range of valid values that can populate the **PV1.PatientClass** field.



3

Edit the invalid data in EDI Explorer.

To return to EDI Explorer and view the **PatientClass** field, either double-click on the error message or expand the message tree until the field is visible. Note that the data associated with this field - A - does not have a match in the HL70004 Validation Table. As a result, the message fails validation.



Edit the message so that the value in the **PatientClass** field matches an entry in the HL70004 Validation Table. Resend the message to Rhapsody. The result should be that an acknowledgement is returned that shows no validation errors are present in the original message.

Lesson 9 of 11

Exercise - Create a CSV Message Definition

The previous exercise described the process of creating a definition for a message format whose specification was held in the EDI Message Designer Library set. This provides a relatively simple way of creating and validating the definition.

In this exercise we examine the process of creating a definition from scratch for a CSV message.

Resources

This exercise uses the messages available in the .\Associate Files and Definitions\109 - Message Definitions\CSV folder in the resources Zip file available at the start of the course.

Review the Message Structure

The first thing we need to do is identify the features of the message structure. It can help to identify these features using the same language we would use when talking about an HL7 message.



This message has the following characteristics:

- The message has no message tag and contains a single, repeating segment.
- Each segment:
 - is NOT identified by a segment tag.
 - uses the comma character to separate each field.
 - is terminated by the [OD][OA] (carriage return & linefeed) characters (the Windows style newline characters).
- Each field may be treated as text (string data type).
- The first segment contains column names. In this definition we will not attempt to process this segment differently.

Definition structure

The message definition comprises one or more of the following:

- **Message Set**
The first set is created when the new definition is created and is named Default. These contain the parameters which define the way the messages in the set are parsed, including the separators and tags, if present.

- **Message**

These contain the parameters for identifying the message and provide a container for the segments which make up the message.

- **Segment**

These contain the parameters for identifying segments and provide a container for the fields which make up the segment.

Optionally, the definition can also include:

- **Composite Fields**

These are a grouping of fields which operate in the same way as a field in a segment. They are used where a common structure is used multiple times throughout the definition (for example, for name, address or date structures).

- **Validation Tables**

These are attached to fields or composites and provide the ability for field content to be validated against a defined data envelope.

These two structures will not be implemented with the CSV message definition created as part of this exercise.

Message Set Configuration

Use the following steps to configure the default message set for a CSV message definition.

1. 1

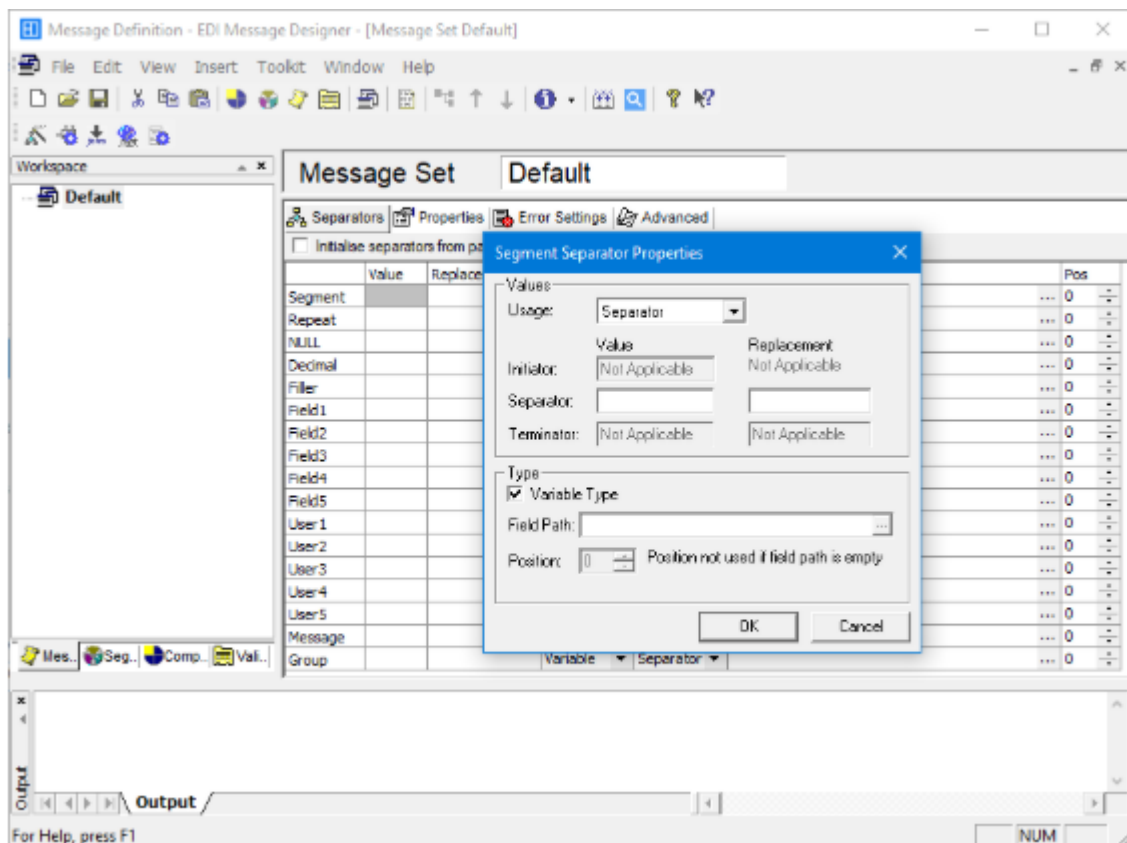
Open EDI Message Designer. Leave the **Message Set** name as *Default*.

2. 2

Double-click the **Message Set** icon in the workspace to open the definition sheet.

3. 3

Ensure the **Separators** tab is selected then double-click the **Value** field for the **Segment** entry.



In the **Segment Separator Properties** dialog:

4

In the **Segment Separator Properties** dialog:

- Change the **Usage** to *Terminator*.
- Enter the string `\x0D\x0A` in the **Separator** field.
This is the hexadecimal representation of the line feed character.
- Clear the **Variable Type** checkbox.
- Select **OK**.

Segment Separator Properties

Values

Usage: Terminator

Value: Not Applicable

Replacement: Not Applicable

Initiator: Not Applicable

Separator: \x0D\x0A

Terminator:

Type

☐ Variable Type

Field Path:

Position: 0

OK Cancel

5

Double-click the **Value** field for the **Field1** entry, where the field separator is defined.

- Ensure the **Usage** is set to *Separator*.
- Enter ',' (comma) in the **Separator** field.
- Clear the **Variable Type** checkbox.

Field1 Separator Properties

Values

Usage: Separator

Value: Not Applicable

Replacement: Not Applicable

Initiator: Not Applicable

Separator: ,

Terminator: Not Applicable

Type

☐ Variable Type

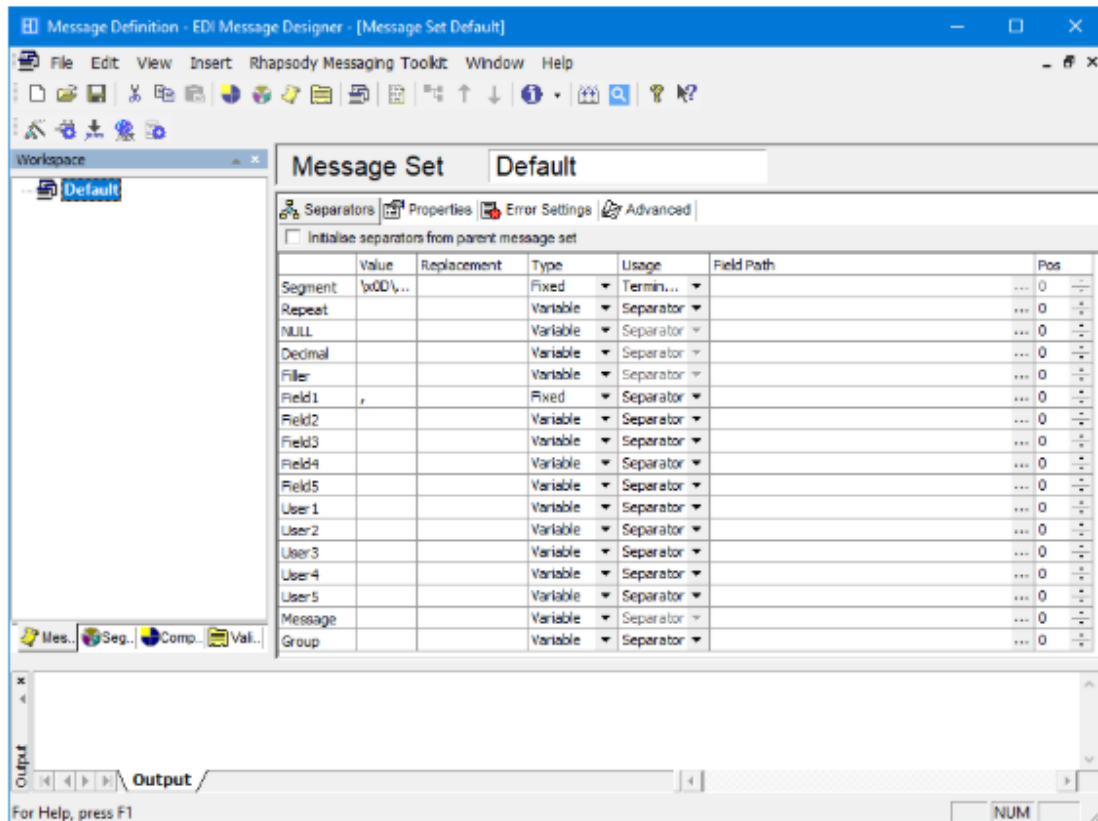
Field Path:

Position: 0

OK Cancel

6

Select **OK**. The resulting configuration will look like this.



Message and Segment Configuration

Now we will name and configure a new CSV message.

1. 1

Create a new message by selecting the **Message** tab in the workspace, then right-clicking and selecting **New Message**. This action opens the **Message** panel with its default **Message Name** highlighted.

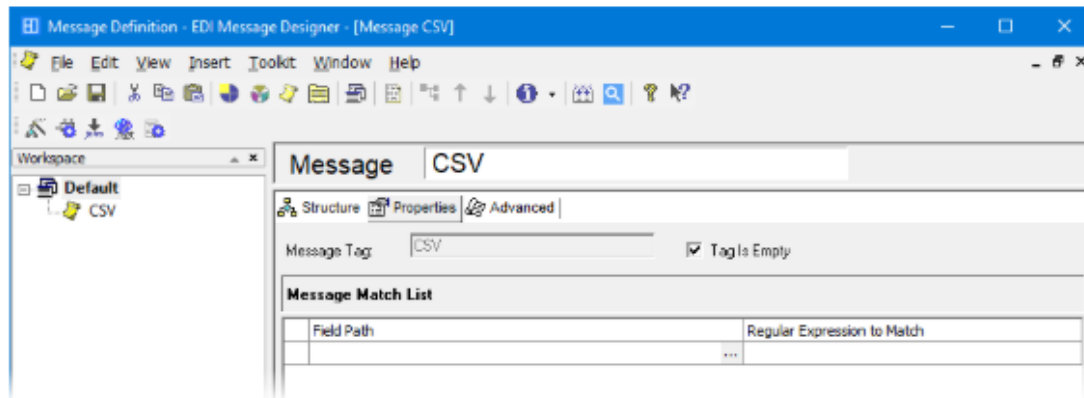
2. 2

Change the message's name to CSV.

The message analysis conducted previously indicates that the message has no identifying tag.

3. 3

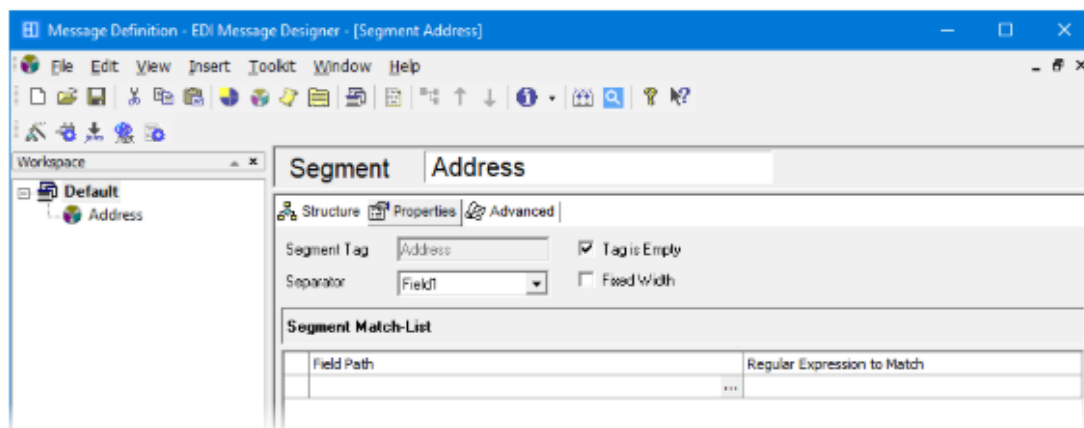
Select the **Properties** tab and ensure the **Tag Is Empty** checkbox is selected.



Add New Segment

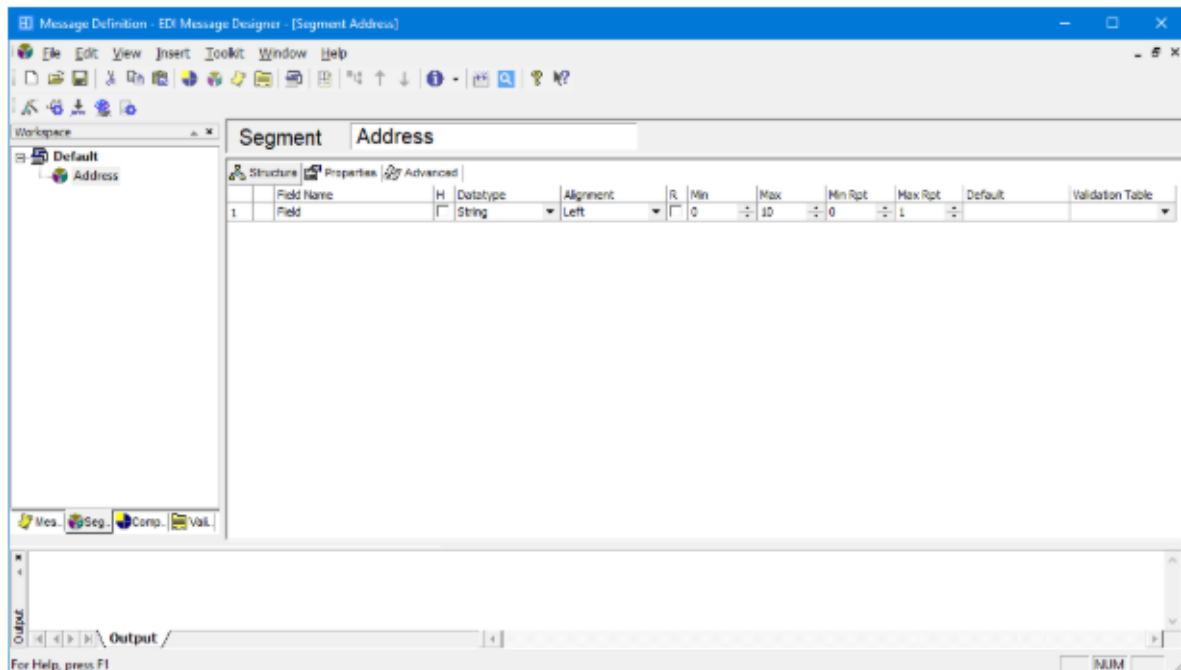
The purpose of this exercise step is to name and configure a new segment in the CSV message definition.

- 1 Create a new segment by selecting the **Segment** tab in the **Workspace** then right-clicking and selecting **New Segment**. This action opens the **Segment** panel with its default **Segment Name** highlighted.
- 2 Change the segment's name to *Address*.
- 3 Select the **Properties** tab and ensure the **Tag Is Empty** checkbox is selected. The **Fixed Width** checkbox should not be selected.



Field Population

Fields are added to the new **Segment** from the **Structure** tab. A stub for the first field was created with the segment. This field name may be changed by double-clicking in the cell and typing the new name, then pressing **Enter**.



New fields may be added by right-clicking in the empty space below the field names and selecting **Insert Field**. Alternatively, use the **Tab** key to traverse the columns; once the last field is tabbed past, a new field will be added.

Create a field structure

Create a field structure including six fields where each field includes the following defaults:

- **Hidden:** *False*
- **Datatype:** *Quoted String*
- **Alignment:** *Left*
- **Required:** *False*
- **Minimum length:** *0*
- **Maximum repeats:** *1*
- **Default value:** *blank*

- **Validation table:** *blank*

The only difference between the fields is their **Field Name** and **Maximum length**:

Field Name	Maximum length
PatientID	20
StreetAddress	30
Suburb	30
City	30
PostCode	10
Country	10

- For the **Hidden** and **Required** attribute settings, the false state is set by leaving the check box un-ticked.
- The **Minimum length** value can be left at 0 (zero) as the field is not required.
- The **Maximum length** value is increased from the default value to allow for longer addresses. The Minimum repeats value is 0 as the field is not required.
- The **Maximum repeats** value is 1 as multiple values for this field are not permitted.

① We are using a datatype of Quoted String as the CSV specification allows a sending system to optionally wrap the contents of a field in double quotes.

If the messages being received did not include a header row, it would be possible to include validation tables against fields that contained a known set of values (such as the country code field).

Add the segment to the message

The definition now has its key elements (the parameters for the Message Set, created and defined a Message and created and defined a Segment). The next step is to associate the Segment with the Message.

1. 1

Select the **Message** tab in the **Workspace** to list the messages in the definition.

2. 2

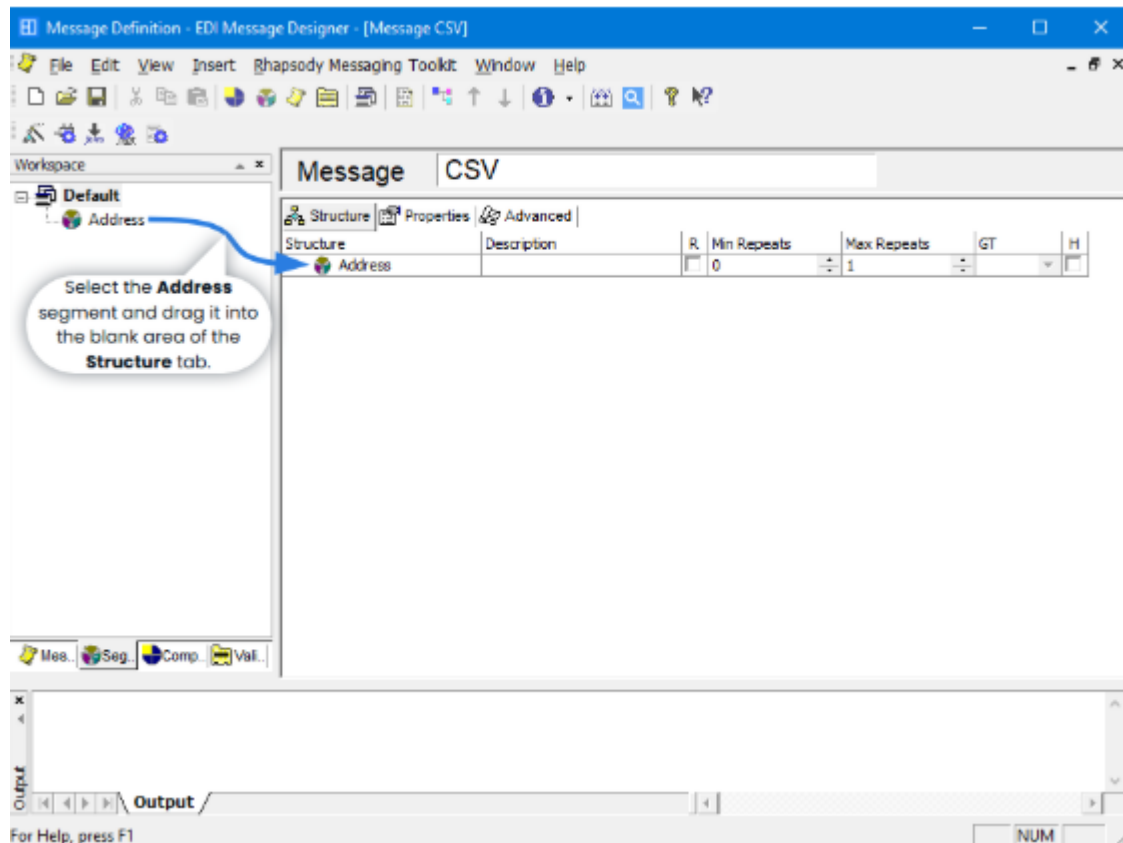
Double-click on the message name to display the details for the CSV message.

3. 3

Select the **Segment** tab in the **Workspace** to list the segments.

4. 4

Select the **Address** segment and drag it into the blank area of the canvas of the **Structure** tab.



1. Select **File > Save** to save the message definition using the file name *CSV_PatientAddress*.

Update message properties

The current configuration for the Address segment sets it as optional with only one repeat in the message. For this exercise, we require the structure to allow multiple occurrences of this segment. To do this, make the following changes:

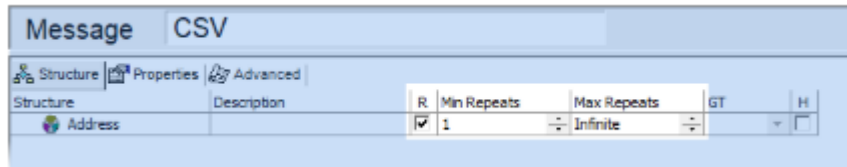
1. 1

Make sure you are viewing the structure for the CSV message.

2. 2

Update the properties for the Address segment in the message as follows:

- **Required:** *Checked*
- **Minimum Repeats:** *1*
- **Maximum Repeats:** *Infinite* (type *inf* and press **Enter**)



1. Save the updated message definition.

Validate the Message Definition in EDI Explorer

We will now validate the CSV message definition against a test message in EDI Explorer.

1. 1

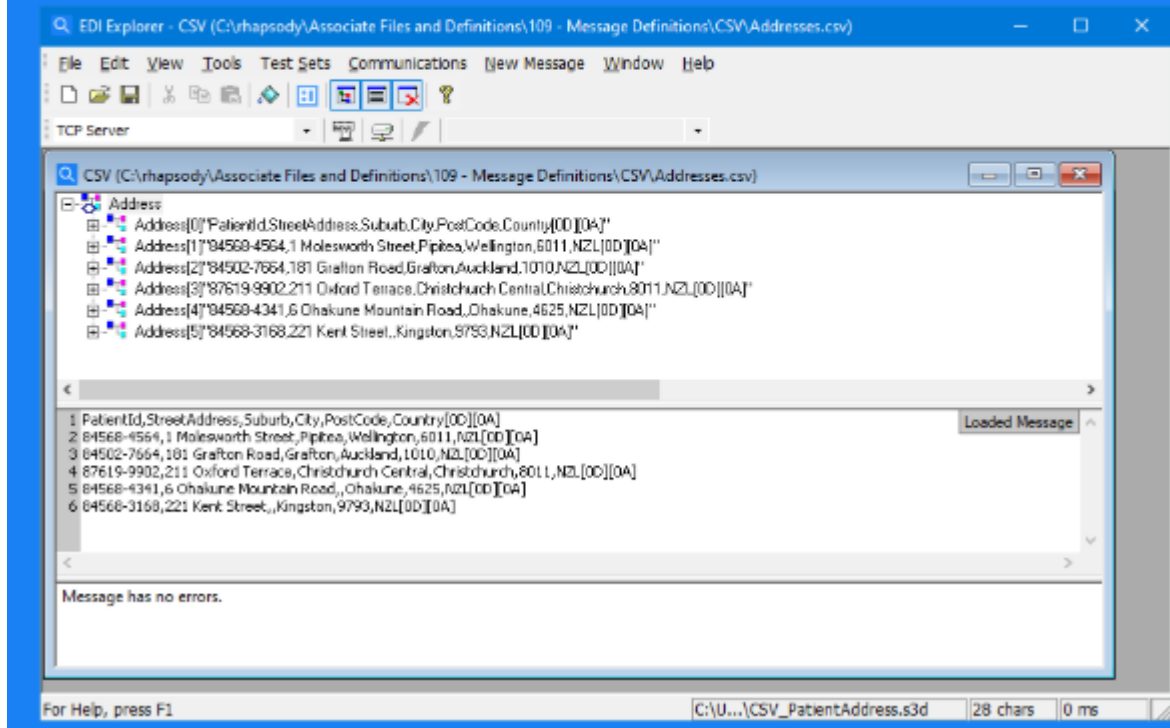
Open EDI Explorer and load it with the newly created CSV message definition. If the definition is already open in EDI Message Designer, select **EDI Explorer** in the Designer's toolbar to open EDI Explorer and load the definition in a single action.

2. 2

Open the Addresses.csv test message located in the exercise resources.

Model Answer

The following screenshot shows the *Addresses.csv* test message in EDI Explorer. It should display without any validation errors.



Validate the Message Definition in Rhapsody

Now, let's upload the CSV message definition to the engine and ensure it parses a test message correctly.

1. 1

Launch Rhapsody IDE. Select the **Definitions** tab and, from the context menu, select **Add Definition > Add Existing Definition**. Select the definition in the **File** dialog.

2. 2

Once the message definition has been loaded, check it in.

3. 3

Create the following route using the parameters listed below the screenshot:



- **Input communication point:** Directory.
- **Filter:** Property Population. On its Properties dialog:

- **Definition File:** the CSV message definition (.s3d) file you created earlier in this exercise.
- **Field->Property Mapping:** Select the **Select (...)** icon to configure this property.

Extract Country Code Properties

General Configuration Message Collector Notes Support Notes Auxiliary Files

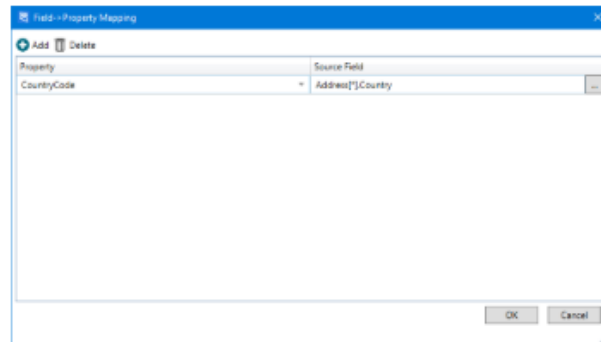
Properties with an asterisk (*) are required. [Test Filter Configuration](#)

Property	Value
Literal->Property Mapping	...
Definition File	CSV_PatientAddress.s3d (New)
Field->Property Mapping	...
On Field Missing *	Continue processing on route

Help OK Cancel

Specify the property **CountryCode** and identify its corresponding source field as shown above.

Entering an asterisk '*' maps all repeats of the Country values from the source message. If specific values are required, note that zero-based counting is used. For example, entering '0' (zero) maps the first repeat, '1' maps the second repeat, and so on.



- **Output communication point:** Sink

- 4 Check in and start all components.
- 5 Place a copy of the *Addresses.csv* test message in the Directory communication point's Input folder.
- 6 Open the Management Console and select the Sink communication point then view its **Output Archive**.
- 7 Select the first listed message to view its details.

Model Answer

The route will extract the value of country code from each of the message segments (including the first segment where its value is 'Country') and set these values as a message property named 'CountryCode'.

The following screenshot shows the *Addresses.csv* test message in the Rhapsody Management Console.

Message View 1.5376.15

< Displaying message 1 of 1 from the result set named 'Message Discard Output Archive' [Back to Result Set](#)

Reject

Redirect Copy

Redirect Edited Copy

Resend

Message Events

Always Show Outputs

Last refreshed at 16:34

COMPONENT	TIMESTAMP	DESCRIPTION
CSV Messages	Oct 6, 2022 16:34:08.164	Received by communication point
CSV Testing	Oct 6, 2022 16:34:08.166	Placed on route
Extract Country Code	Oct 6, 2022 16:34:08.169	Filtering succeeded; took 2.747 ms
Message Discard	Oct 6, 2022 16:34:08.169	Delivered to communication point
	Oct 6, 2022 16:34:13.331	Sent by communication point, total pro...

Message Properties

Property Name	Property Value
BaseFilename	Addresses
CountryCode	[Country,NZL,NZL,NZL,NZL]
DefinitionName	CSV_PatientAddress.s3d
directory:BodySize	347
FileCreationTime	1665023037000
FullFilename	Addresses.csv
InputTime	1665027248164
rhapsody:inputCommunicationPoint	CSV Messages
rhapsody:inputCommunicationPointId	63
rhapsody:OutputMessageType	CSV

Show More

Message Body

Tree View

Flat Wire

Show Whitespace Encoding: US-ASCII

Size: 347 bytes

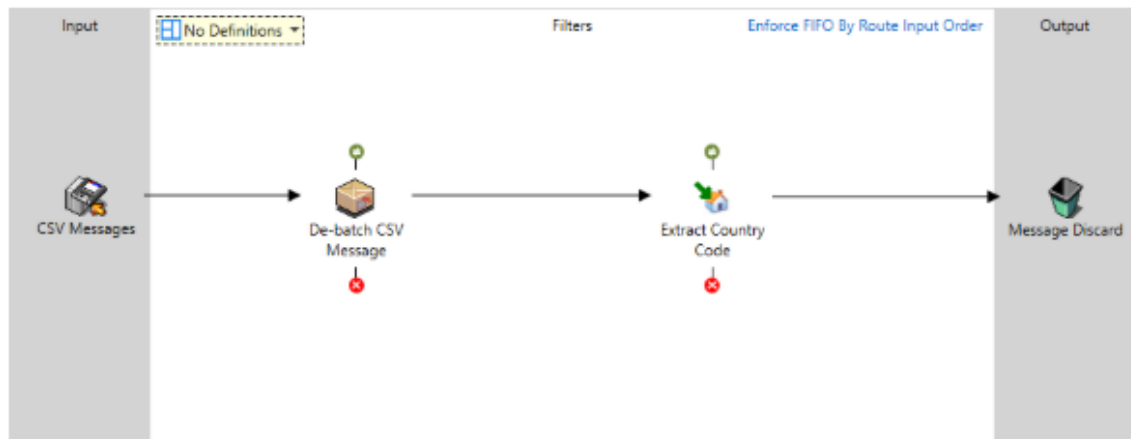
PatientId,StreetAddress,Suburb,City,PostCode,Country\r\n84568-4564,1 Molesworth Street,Pipitea,Wellington,6011,NZL\r\n84502-7664,99 Khyber Pass Road,Grafton,Auckland,1010,NZL\r\n87619-9980,211 Oxford Terrace,Christchurch Central,Christchurch,8011,NZL\r\n84568-4341,6 Ohakune Mountain Road,,Ohakune,4625,NZL\r\n84568-3268,221 Kent Street,,Kingston,9793,NZL\r\n

Download

De-batch a Test Message using the Directory Communication Point

1

Modify the route to add a Batch / De-Batch filter before the existing Property Population filter. We will configure this filter to split the incoming file at the segment separator.



2

The **Mode** property should be set to De-batch and the **Message Separator Expression** property set as `\x0D\x0A`. This matches the segment terminator expression we used in the definition.

Property	Value
Mode *	De-batch
Header	
Separator	
Trailer	
Batch Header Expression	
Batch Trailer Expression	
Separator Type *	Separates messages
Message Separator Expression *	\x0D\x0A
Set Identifier Property	
Index Base *	Zero

3

Test the route with the test message and examine the output in the Management Console.

Model Answer

This route will de-batch the *Addresses.csv* input message into six separate messages - one for each of its segments. Five messages will contain patient address details, while the sixth will include the titles from the first segment.

The following screenshot shows a de-batched *Addresses.csv* message containing the address information for one of the patients.

Message View 1.10496.36

« Displaying message 6 of 7 from the result set named 'Message Discard Output Archive (2)' [Back to Result Set](#)

[Reinject](#)
[Redirect Copy](#)
[Redirect Edited Copy](#)
[Resend](#)

Message Events Always Show Outputs Last refreshed at 16:37

COMPONENT	TIMESTAMP	DESCRIPTION
CSV Messages	Oct 6, 2022 16:37:14.474	Received by communication point
CSV Testing	Oct 6, 2022 16:37:14.476	Placed on route
De-batch CSV Message	Oct 6, 2022 16:37:14.487	Filtering succeeded, took 10.836 ms
Extract Country Code	Oct 6, 2022 16:37:14.492	Filtering succeeded, took 0.681 ms
Message Discard	Oct 6, 2022 16:37:14.495	Delivered to communication point
Extract Country Code	Oct 6, 2022 16:37:14.496	Sent by communication point, total pro...

« « « » » »

Message Properties

Property Name	Property Value
BaseFilename	Addresses
BatchIndex	1
CountryCode	NZL
DefinitionName	CSV_PatientAddress.s3d
directoryBodySize	347
FileCreationTime	1665020596000
FullFilename	Addresses.csv
InputTime	1665027434474
rhapsodyInputCommunicationPoint	CSV Messages
rhapsodyInputCommunicationPointId	63

[Show More](#)

Showing 10 of 13

Message Body

Tree View Flat View

Show Whitespace Encoding: US-ASCII Size: 58 bytes

```
84568-4564,1 Molesworth Street,Pipitea,Wellington,6011,NZL
```

[Download](#)

Exercise - Build an XML Message Definition

Rhapsody provides support for XML messages in two ways:

- Implicitly by means of XPath references in the JavaScript filter.
- Explicitly by means of an XSD (XML Schema Definition).

Complex XML messages will commonly have an associated XSD. It is sometimes necessary, however, to build an XSD for simple structures. The Rhapsody IDE includes an XSD generator to assist in the process of generating the XSD.

In this exercise we will use a sample XML message to build an XSD for the message.

Resources

This exercise makes use of the messages available in the in the *.\Associate Files and Definitions\109 - Message Definition\XML* folder in the resources Zip file available at the start of the course.

Create an XSD File

From the IDE Tools menu, select **XSD Generator** then complete the following three-step process to create the file.

1. 1

Select **Load XML Documents** and select the three sample XML messages. Select a XML message to view its contents in the right-hand panel.

2. 2

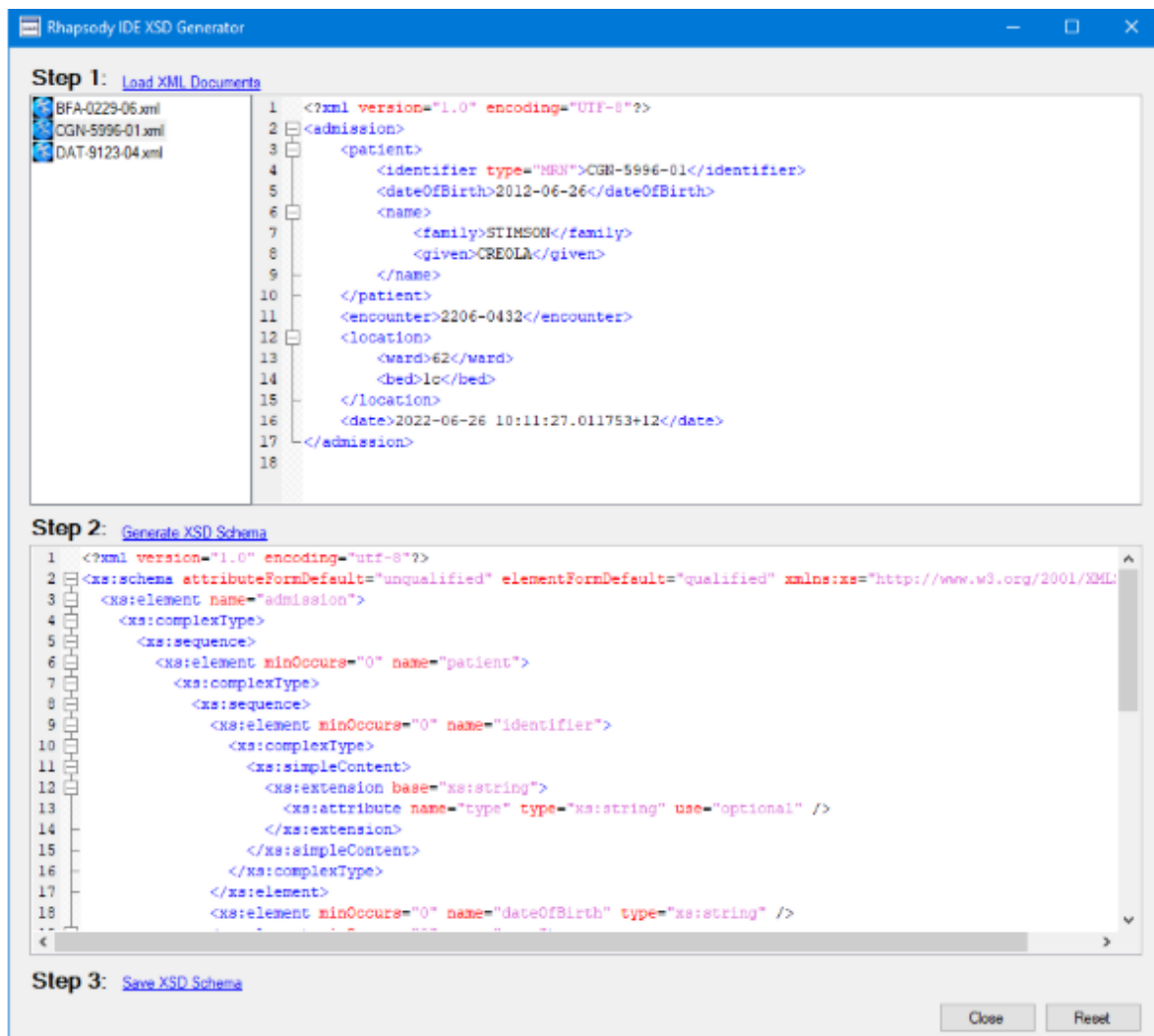
Select **Generate XSD Schema**. The schema will be created and displayed in the "Step 2" window.

3. 3

Select **Save XSD Schema** to save the schema, using the filename that describes the content of the of the XML format. For example: *patient-admission.xsd*.

4. 4

Close the XSD Generator.



Open the saved file in the text editor of your choice.

Model Answer

The XSD file created using the Rhapsody IDE XSD Generator will appear as shown below:

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<xs:schema attributeFormDefault="unqualified" elementFormDefault="qualified"
xmlns:xs="https://www.w3.org/2001/XMLSchema">
```

```
<xs:element name="message">
```

```
<xs:complexType>
```

```
<xs:sequence>
```

```
<xs:element minOccurs="0" name="recent">
```



```

<xs:complexType>
  <xs:sequence>
    <xs:element minOccurs="0" name="pi_index" type="xs:string" />
    <xs:element minOccurs="0" name="source_file">
      <xs:complexType>
        <xs:attribute name="null" type="xs:string" use="optional" />
      </xs:complexType>
    </xs:element>
    <xs:element minOccurs="0" name="modified_date" type="xs:string" />
    <xs:element minOccurs="0" name="modified_by" type="xs:string" />
    <xs:element minOccurs="0" name="last_name" type="xs:string" />
    <xs:element minOccurs="0" name="id_org" type="xs:string" />
    <xs:element minOccurs="0" name="patient_id" type="xs:string" />
    <xs:element minOccurs="0" name="given_name" type="xs:string" />
    <xs:element minOccurs="0" name="lastaccess" type="xs:string" />
  </xs:sequence>
</xs:complexType>
</xs:element>
</xs:sequence>
</xs:complexType>
</xs:element>
</xs:schema>

```

Import the XSD Schema

An XSD schema is used in the same manner as other message definition types. They must be:

- Uploaded and checked in to the Rhapsody engine.
- Loaded on to the communication points and/or filter Configuration dialogs that require it.

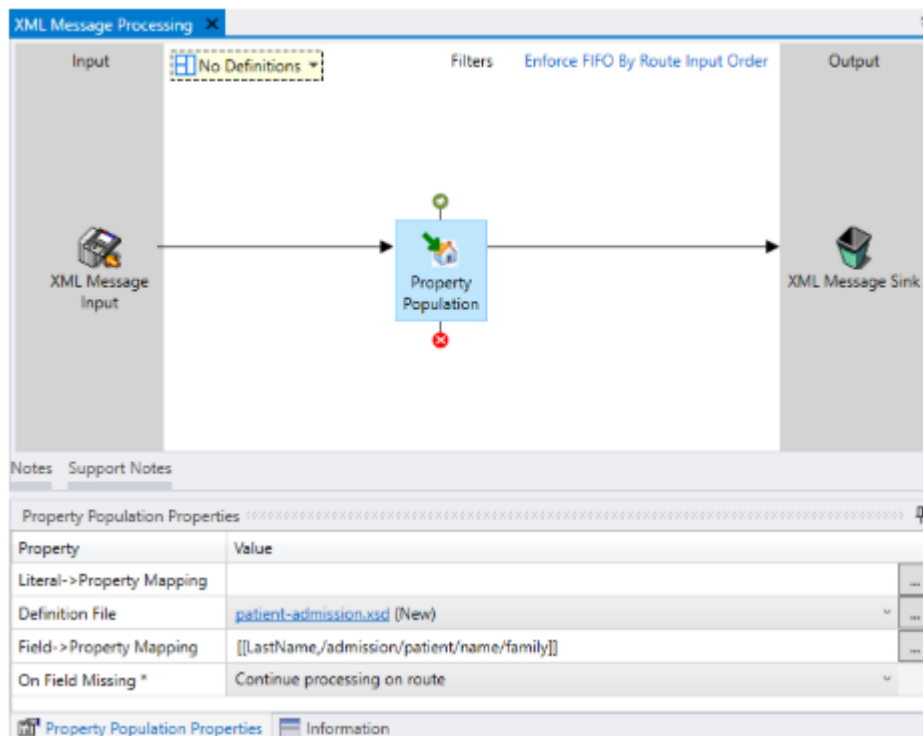
Open the **Definitions** tab in the IDE **Workspace**. From the context menu, select **Add Definition > Add Existing Definition**.

1. 1

Select the XSD schema you created in the **File** dialog and select **Open**. Check in the definition.

2. 2

Create a route similar to the one below to test the XSD schema and its parsing ability:



3

Configure **Field->Property Mapping** property of the Property Population filter to add a message property called **LastName** and populate it with the `/admission/patient/name/family` field in the xml message. Use the field picker to select the value.

4

Ensure all components are checked in and started.

Test the route

Place a copy of the *CGN-5996-01.xml* test message into the Directory communication point's input folder. Open the Management Console and examine the message as it is processed by the Property Population filter. Use the **Compare In and Out** view to highlight the effect of the filter on the message.

Test the route

Place a copy of the *CGN-5996-01.xml* test message into the Directory communication point's input folder. Open the Management Console and examine the message as it is processed by the Property Population filter. Use the **Compare In and Out** view to highlight the effect of the filter on the message.

Model Answer

The following screenshot shows a message following processing by the Property Population filter:

Message Events
Always Show Outputs ☒ Last refreshed at 15:40

COMPONENT	TIMESTAMP	DESCRIPTION
XML Message Input	Aug 8, 2022 15:38:50.177	Received by communication point
XML Message Processing	Aug 8, 2022 15:38:50.191	Placed on route
Property Population	Aug 8, 2022 15:38:50.318	Filtering succeeded; took 53.203 ms
XML Message Sink	Aug 8, 2022 15:38:50.329	Delivered to communication point
	Aug 8, 2022 15:38:50.338	Sent by communication point, total pro...

Message Properties

In

Property Name	Property Value
BaseFilename	CGN-5996-01
directory:BodySize	458
FileCreationTime	1659929200000
FullFilename	CGN-5996-01.xml
InputTime	1659929930175
rhapsody:InputCommunicationPoint	XML Message Input
rhapsody:InputCommunicationPoin...	5
Suffix	.xml

Showing 8 of 8

Out

Property Name	Property Value
BaseFilename	CGN-5996-01
DefinitionName	patient-admission.xsd
directory:BodySize	458
FileCreationTime	1659929200000
FullFilename	CGN-5996-01.xml
InputTime	1659929930175
LastName	STIMSON
rhapsody:InputCommunicationPoint	XML Message Input
rhapsody:InputCommunicationPoin...	5
rhapsody:OutputMessageType	admission

[Show More](#)
Showing 10 of 11

Message Body

Tree View

Flat Wire

In
XML Encoding: US-ASCII Size: 458 bytes

```
<dateOfBirth>
2012-06-26
</dateOfBirth>
<name>
  <family>
    STIMSON
  </family>
  <given>
    CREOLA
  </given>
</name>
```

Download

Out
XML Encoding: US-ASCII Size: 458 bytes

```
<admission>
  <patient>
    <identifier type="MRN">
      CGN-5996-01
    </identifier>
    <dateOfBirth>
      2012-06-26
    </dateOfBirth>
    <name>
      <family>
```

Download